DATA STRUCTURES AND ALGORITHMS
23CSE203

Submitted by	
Name	Ch.Sri Vasavi
Roll No	AV.SC.U4CSE25026
Year/SEM/Section	2nd Year/SEM-3/CSE-A
Credits	5

Submitted to	
Name	DR.RAJ KUMAR BATCHU
Department	School of Computing
Designation	Assistant Professor (SI. Grade)

INDEX

SNO	TITLE	PageNo	Date
1.	**WEEK-1:** 1A) A Rocket launch system displays a countdown before launch. Instead of suing a loop, implement the countdown using recursion. Write a recursive python program to display a countdown from n to 1 followed by "launch". 1B) Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate p(n) where p=principal growth, n= no. of years. 1C) An Hr department stores employee IDs in a list. Develop a recursive program to search for a given employee ID write a recursive function to search for an employee ID in a list. 1D) In a logistics company, the no .of possible ways to arrange in different parcels is calculated using the factorial formula (n!). Develop a recursive program to calculate the factorial of a given number. 1E) Write a recursive program to print the first n terms of the Fibonacci series.	4-13	23-07-26
2.	**WEEK 2:** 2A) Write a python program to perform linear search on the give input. 2B) Write a python program to perform binary search on the given. Verify by taking sorted array as an input. 2c) Write a python program to perform binary search on the given. Verify by taking unsorted array as an input.	14-21	30-07-26
3.	**WEEK-3:** 3a) write a python program to perform bubble sort on the given input. 3b) write a python program to perform insertion sort on the given input. 3c) write a python program to perform selection sort by taking given input. 3d) write a python program to perform merge sort by taking the given input. 3e) write a python program to perform quick sort by taking given input.	22-35	06-08-26

4.	**Week-4**: 4a) write a python program to perform linked list operations by taking the input from the user 4b) write a python program to perform double linked list operations by taking input from the user 4c) write a python program to perform circular linked list operations by taking input from the user	36-60	13-08-26
5.	**Week-5:** 5a) write a python program to implement the stack by using arrays. 5b) write a python program to implement stack using the linked list	61-72	27-08-26
6.	**Week-6:** 6a) write a python program to implement the queue by suing the arrays. 6b) write a python program to implement the queue by using the linked list. 6c) write a python program to implement the circular queue using the arrays	73-89	03-09-26
7.	**Week-7:** 7a) write a python program to implement the binary tree using arrays 7b) write a python program to implement the binary tree using linked list	90-107	24-09-26
8.	**Week-8:** 8a) write a python program to implement the binary search tree using the arrays	108-116	08-10-26
9.	**Week-9:** Write a python program to implement the AVL tree using arrays	117-124	22-10-26
10.	**Week-10:** Write a python program on hashing separate Chaining, linear, quadratic probing	125-133	29-10-26

WEEK-1

1A) **AIM:** A Rocket launch system displays a countdown before launch. Instead of suing a loop, implement the countdown using recursion. Write a recursive python program to display a countdown from n to 1 followed by “launch”.

Code:

1A).py - C:/Users/Srivasavi/OneDrive/Documents/CSE25026_DSA/WEEK-1/1A).py (3.13.14)

File Edit Format Run Options Window Help

```
def launch(n):
    if n==0:
        print("Launch")
        return
    print(n)
    launch(n-1)
n=int(input("Enter the duration of the Countdown : "))
launch(n)

#Alternate
"""
def launch(n):
    while n>0:
        return n
    else:
        return "Launch"
n=int(input("Enter Number : "))
while n>=0:
    print(launch(n))
    n-=1
"""
```

Output:

IDLE Shell 3.13.14

File Edit Shell Debug Options Window Help

```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
== RESTART: C:/Users/Srivasavi/OneDrive/Documents/CSE25026_DSA/WEEK-1/1A).py =
Enter the duration of the Countdown : 10
10
9
8
7
6
5
4
3
2
1
Launch
```

Explanation

Condition 1: Positive Input (n > 0)

- The function checks whether n == 0.
- Since n is positive, the condition is **False**.
- It executes:
  - print(n) → prints the current value of n.
  - countdown(n-1) → calls the same function again with n-1.
- This process continues until n becomes 0.
- When n == 0, the condition becomes **True**.
- It prints **launch**.
- The return statement stops that function call.
- Then the recursive calls return one by one.
- Since there is no value returned by the function, it returns **None**.

Condition 2: Zero Input (n == 0)

- The function checks if n == 0.
- The condition is immediately **True**.
- It prints **launch**.
- The return statement stops the function.
- No recursive call takes place.
- The function returns **None**.

Condition 3: Negative Input (n < 0)

- The condition n == 0 is **False**.
- It prints the negative value.
- Then it calls itself with n-1, making the value even more negative.
- Therefore, it will **never reach 0**.
- The recursion continues indefinitely.

- Eventually, Python raises a **RecursionError** because the maximum recursion depth is exceeded

Error table:

SNO	Error obtained	Rectification
1.	Wrong return Statement caused infininte loop	Corrected the return statement

1B) **AIM**: Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate p(n) where p=principal growth, n= no. of years.

Code:

1B).py - C:/Users/Srivasavi/OneDrive/Documents/CSE25026_DSA/WEEK-1/1B).py (3.13.14)

File Edit Format Run Options Window Help

```
def intrest(p,n):
    if n==0:
        return 1
    return p*intrest(p,n-1)
p=int(input("Enter Principal amount : "))
n=float(input("Enter no of years : "))
print(f"The Simple Intrest in {n} years is ",intrest(p,n))
```

Output:

IDLE Shell 3.13.14

File Edit Shell Debug Options Window Help

```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
== RESTART: C:/Users/Srivasavi/OneDrive/Documents/CSE25026_DSA/WEEK-1/1B).py =
Enter Principal amount : 25000
Enter no of years : 5
The Simple Intrest in 5.0 years is  9765625000000000000000
```

Explanation

Condition 1: n > 0

- The function compound_interest(p, n) is called with the principal amount p and number of years n.
- The function checks whether n == 0.
- Since n is greater than 0, the condition is **False**.
- It executes the else block.
- It calculates:
  - p * compound_interest(p, n-1)
- The function calls itself recursively with n-1.
- This process continues until n becomes 0.
- Each recursive call multiplies p one more time.
- Therefore, the function calculates $\mathbf{p^n}$.

Condition 2: n == 0

- When n becomes 0, the condition n == 0 becomes **True**.
- The function returns 1.
- This 1 acts as the **base case** for the multiplication.
- The recursive calls then return one by one.
- Each returned value is multiplied by p.
- Finally, the result is printed using print(compound_interest(p,n))

Error table:

s.no	Error obtained	rectification

1	Wrong indentation	Maintain proper indentation.

1C) **AIM** : An Hr department stores employee IDs in a list .Develop a recursive program to search for a given employee ID write a recursive function to search for an employee ID in a list.

Code:

1C).py - C:\Users\Srivasavi\OneDrive\Documents\CSE25026_DSA\WEEK-1\1C).py (3.13.14)

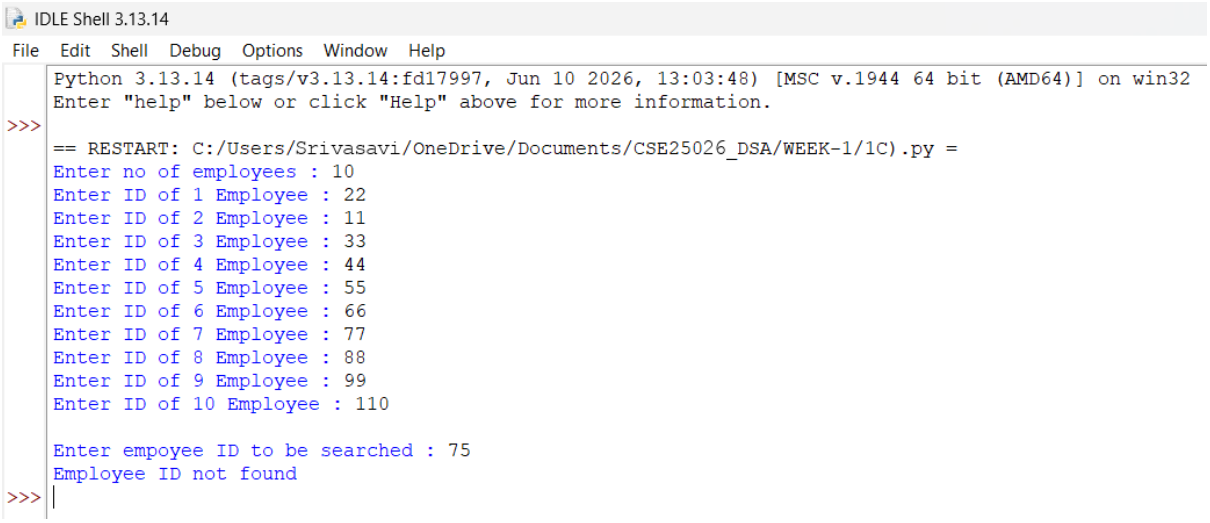
File Edit Format Run Options Window Help

```
def search(target,arr,i):
    if i<len(arr):
        if target==arr[i]:
            return i
        else:
            return search(target,arr,i+1)
    return -1

n=int(input("Enter no of employees : "))
arr=[]
for i in range(n):
    a=int(input(f"Enter ID of {i+1} Employee : "))
    arr.append(a)
print()
target=int(input("Enter empoyee ID to be searched : "))
s=search(target,arr,0)
if s==-1:
    print("Employee ID not found")
else:
    print("Employee ID found at ",s)

"""
ALternative Method:
def search(l,h,target,arr):
    while l<=h:
        m=(l+h)//2
        if arr[m]==target:
            return m
        elif arr[m]<target:
            l=m+1
        else:
            h=m-1
    return -1
arr.sort()
"""
```

Output:



Explanation

Condition 1: index == len(emp_list)

- The function checks whether index has reached the length of the employee list.
- If index == len(emp_list), it means we have checked **all employee IDs**.
- The employee ID was not found.
- The function returns **False**.
- No further recursion takes place.

Condition 2: emp_list[index] == emp_id

- The function checks whether the current employee ID is equal to the ID we are searching for.
- If both IDs are equal, the condition becomes **True**.
- The function returns **True**.
- The recursion stops because the employee ID has been found.

Condition 3: Employee ID does not match

- If the current employee ID is not equal to emp_id, the condition is **False**.
- The function calls itself recursively with:
  - index + 1
- This moves to the **next employee ID** in the list.
- The process continues until:
  - The employee ID is found, or
  - The entire list is checked.

Error table:

s.no	Error obtained	Rectification
1	Missing return statement	Add the required return statement

1D) **AIM**.: In a logistics company , the no . of possible ways to arrange in different parcels is calculated using the factorial formula (n!). Develop a recursive program to calculate the factorial of a given number.

Code:

1D).py - C:/Users/Srivasavi/OneDrive/Documents/CSE25026_DSA/WEEK-1/1D).py (3.13.14)

File Edit Format Run Options Window Help

```
def fact(n):
    if n==0 or n==1:
        return 1
    return n*fact(n-1)
a=int(input("Enter the Number : "))
print(f"Factorial of {a} is ",fact(a))
```

Output:

IDLE Shell 3.13.14

File Edit Shell Debug Options Window Help

```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
== RESTART: C:/Users/Srivasavi/OneDrive/Documents/CSE25026_DSA/WEEK-1/1D).py =
Enter the Number : 10
Factorial of 10 is  3628800
>>>
```

Explanation

Condition 1: Negative Input (n < 0)

- The program first checks whether n < 0.
- If n is negative, the condition is **True**.
- Factorial is not defined for negative numbers.
- It prints:
  Factorial is not defined for negative numbers.
- The factorial() function is **not called**.

Condition 2: Base Case (n == 0 or n == 1)

- Inside the factorial() function, it checks:
  n == 0 or n == 1
- If n is 0 or 1, the condition is **True**.
- The function returns **1**.
- This is the **base case** of the recursion.
- It stops further recursive calls.

Condition 3: n > 1

- If n is greater than 1, the condition n == 0 or n == 1 is **False**.
- The else block is executed.
- It returns:
  n * factorial(n - 1)
- The function calls itself with n - 1.
- This continues until n becomes 1.
- Then the returned values are multiplied together

Error table:

s.no	Error obtained	Rectification
1	Wrong recursive value	Updated the recursive value correctly

1E) **AIM**: Write a recursive program to print the first n terms of the Fibonacci series

Code:

1E).py - C:/Users/Srivasavi/OneDrive/Documents/CSE25026_DSA/WEEK-1/1E).py (3.13.14)

File Edit Format Run Options Window Help

```
def fibo(n):
    if n==0:
        return 0
    elif n==1:
        return 1
    return fibo(n-1)+fibo(n-2)
a=int(input("Enter a Number of Terms : "))
for i in range(a):
    print(fibo(i),end=" ")
print()
print(f"The {a}th element of Fibonacci series is ",fibo(a-1))
```

Output:

IDLE Shell 3.13.14

File Edit Shell Debug Options Window Help

```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
== RESTART: C:/Users/Srivasavi/OneDrive/Documents/CSE25026_DSA/WEEK-1/1E).py =
Enter a Number of Terms : 20
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987 1597 2584 4181
The 20th element of Fibonacci series is  4181
>>>
```

Explanation

Condition 1: n == 0

- The function checks whether n == 0.
- If the condition is **True**, the function returns 0.
- This is the **first base case**.
- No further recursive call takes place.

Condition 2: n == 1

- If n is not 0, the function checks whether n == 1.
- If the condition is **True**, the function returns 1.
- This is the **second base case**.
- No further recursive call takes place

Condition 3: n > 1

- If n is neither 0 nor 1, both conditions are **False**.
- The else block is executed.

- The function calls itself **two times**:
  - fibonacci(n - 1)
  - fibonacci(n - 2)
- The two returned values are added together.
- This process continues until the base cases n == 0 and n == 1 are reached.

Error table:

s.no	Error obtained	Rectification
1	Incorrect Fibonacci recursion	Use fibonacci(n-1) + fibonacci(n-2).

WEEK-2

2A) **AIM:** Write a python program to perform linear search on the give input.

Code:

Linear.py - C:\Users\Srivasavi\OneDrive\Documents\CSE25026_DSA\WEEK-2\Linear.py (3.13.14)

File Edit Format Run Options Window Help

```
def search(target,arr):
    i=0
    while i<len(arr):
        if arr[i]==target:
            return i
        i+=1
    return -1

a=input("Enter Elelments in Array : ")
arr=[int(i) for i in a.split()]

target=int(input("Enter Element to be searched : "))
s=search(target,arr)
if s==-1:
    print("Element not found")
else:
    print("Element found at index : ",s)
```

Output:

IDLE Shell 3.13.14

File Edit Shell Debug Options Window Help

```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:\Users\Srivasavi\OneDrive\Documents\CSE25026_DSA\WEEK-2\Linear.py
Enter Elelments in Array : 20 30 4 5 7 10 66 53 78 90
Enter Element to be searched : 7
Element found at index :  4
>>>
========== RESTART: C:\Users\Srivasavi\OneDrive\Documents\CSE25026_DSA\WEEK-2\Linear.py ==========
Enter Elelments in Array : 20 30 4 5 7 10 66 53 78 90
Enter Element to be searched : 78
Element found at index :  8
>>>
========== RESTART: C:\Users\Srivasavi\OneDrive\Documents\CSE25026_DSA\WEEK-2\Linear.py ==========
Enter Elelments in Array : 20 30 4 5 7 10 66 53 78 90
Enter Element to be searched : 1
Element not found
>>>
```

Explanation

Condition 1: Element is Found (arr[i] == k)

- The function linear_search(arr, k) receives the array arr and the search element k.
- The for loop starts from the first element of the array.

- It checks each element one by one using:
  arr[i] == k
- If the current element is equal to k, the condition becomes **True**.
- The function immediately returns the **index i**.
- Since the function returns the index, the loop stops immediately.
- The remaining elements are not checked.

Condition 2: Element is Not Found

- The loop checks every element in the array.
- If none of the elements is equal to k, the if condition is always **False**.
- After the loop finishes, the statement return -1 is executed.
- -1 indicates that the search element is **not present** in the array.

Error table:

s.no	Error obtained	Rectification
1	Wrong loop range	Use range(n) to process exactly n elements.

2B) **AIM:** Write a python program to perform binary search on the given. Verify by taking sorted array as an input.

Code:

Binary_sorted.py - C:\Users\Srivasavi\OneDrive\Documents\CSE25026_DSA\WEEK-2\Binary_sorted.py (3.13.14)

File Edit Format Run Options Window Help

```
def search(l,h,target,arr):
    while l<=h:
        m=(l+h)//2
        if arr[m]==target:
            return m
        elif arr[m]<target:
            l=m+1
        else:
            h=m-1
    return -1

a=input("Enter Elelments in Array in Increasing Order : ")
arr=[int(i) for i in a.split()]

target=int(input("Enter Element to be searched : "))
if arr != arr.sorted():
    print("Enter elements in sorted order")
    exit()
s=search(0,len(arr)-1,target,arr)
if s==-1:
    print("Element not found")
else:
    print("Element found at index : ",s)
```

Output:

IDLE Shell 3.13.14

File Edit Shell Debug Options Window Help

```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:\Users\Srivasavi\OneDrive\Documents\CSE25026_DSA\WEEK-2\Binary_sorted.py
Enter Elelments in Array in Increasing Order : 1 2 3 45 88 990 8888 9999
Enter Element to be searched : 3
Element found at index :  2
>>>
= RESTART: C:\Users\Srivasavi\OneDrive\Documents\CSE25026_DSA\WEEK-2\Binary_sorted.py
Enter Elelments in Array in Increasing Order : 1 2 3 45 88 990 8888 9999
Enter Element to be searched : 8888
Element found at index :  6
>>>
= RESTART: C:\Users\Srivasavi\OneDrive\Documents\CSE25026_DSA\WEEK-2\Binary_sorted.py
Enter Elelments in Array in Increasing Order : 7
Enter Element to be searched : 1
Element not found
>>>
```

Explanation

Condition 1: k == arr[mid] — Element Found

- The function binary_search(arr, k) receives the array and the search element.
- low is initialized to 0, which represents the first index.

- high is initialized to len(arr)-1, which represents the last index.
- The while loop runs as long as low <= high.
- mid is calculated using:
  mid = (low + high) // 2
- The middle element is compared with the search element k.
- If k == arr[mid], the element is found.
- The function returns the **index mid** and stops searching.

Condition 2: k < arr[mid] — Search Left Half

- If k is smaller than the middle element, the condition is **True**.
- This means the search element can only be present in the **left half**.
- Therefore:
  high = mid - 1
- The right half is eliminated.
- The process continues with the remaining left half

Condition 3: k > arr[mid] — Search Right Half

- If k is greater than the middle element, the elif condition is **False**.
- The else block is executed.
- This means the search element can only be present in the **right half**.
- Therefore:
  low = mid + 1
- The left half is eliminated.
- The process continues with the remaining right half.

Condition 4: low > high — Element Not Found

- The while loop continues until low <= high becomes **False**.
- When low > high, there are no elements left to search.
- This means the search element is not present in the array.
- The function returns **-1**.

Error table:

s.no	Error obtained	Rectification
	Wrong index in binary search	Use high = mid - 1 for the left half and low = mid + 1 for the right half.

2C)**AIM:** Write a python program to perform binary search on the given. Verify by taking unsorted array as an input.

Code:

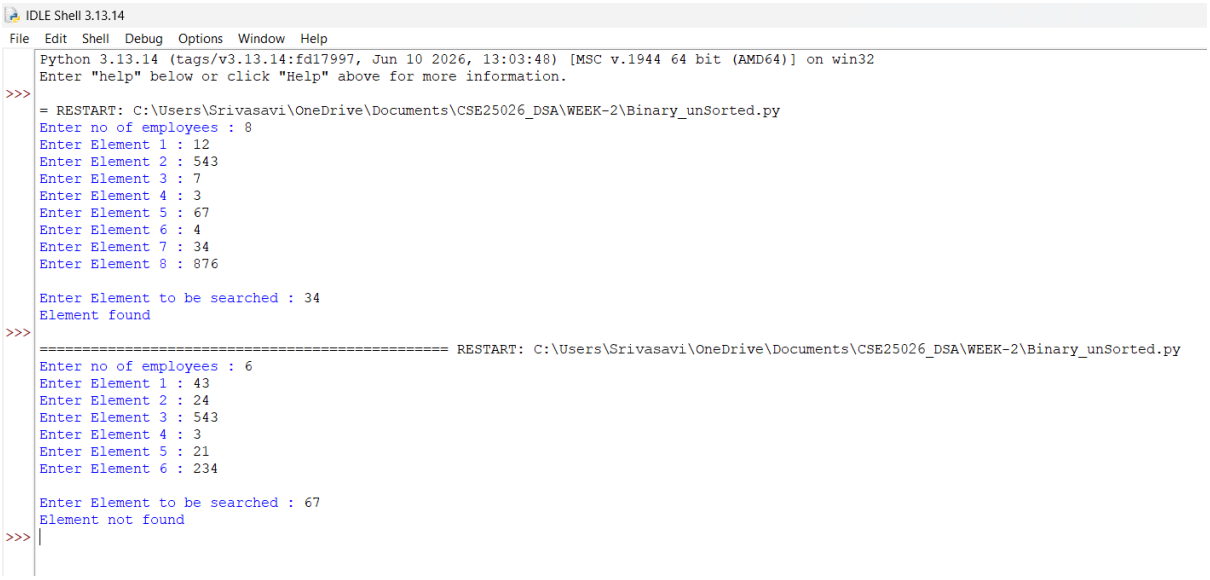
Binary_unSorted.py - C:\Users\Srivasavi\OneDrive\Documents\CSE25026_DSA\WEEK-2\Binary_unSorted.py (3.13.14)

File Edit Format Run Options Window Help

```
def search(l,h,target,arr):
    while l<=h:
        m=(l+h)//2
        if arr[m]==target:
            return m
        elif arr[m]<target:
            l=m+1
        else:
            h=m-1
    return -1

n=int(input("Enter no of employees : "))
arr=[]
for i in range(n):
    a=int(input(f"Enter Element {i+1} : "))
    arr.append(a)
print()
arr.sort()
target=int(input("Enter Element to be searched : "))
s=search(0,len(arr)-1,target,arr)
if s==-1:
    print("Element not found")
else:
    print("Element found")
```

Output:



Explanation

Condition 1: arr[mid] == key — Element Found

- The function first calculates the middle index using mid = (low + high) // 2.
- It compares the middle element arr[mid] with the search element key.
- If arr[mid] == key, the condition is **True**.
- The element is found.
- The function returns the index mid.
- The search stops.

Condition 2: key < arr[mid] — Search Left Half

- If key is smaller than arr[mid], the condition is **True**.
- This means the required element may be present in the **left half**.
- The right half is ignored.
- high is changed to mid - 1.
- The search continues in the left half.

Condition 3: key > arr[mid] — Search Right Half

- If key is greater than arr[mid], the previous conditions are **False**.

- The element may be present in the **right half**.
- The left half is ignored.
- low is changed to mid + 1.
- The search continues in the right half.

Condition 4: low > high — Element Not Found

- The while low <= high loop continues until there are elements left to search.
- When low > high, the loop stops.
- This means the element is not present in the array.
- The function returns -1.
- The program prints **"Element not found."**

Condition 5: Array is Already Sorted

- The program checks whether the given array is already sorted.
- It uses:
  arr == sorted(arr)
- If the condition is **True**, it prints:
  "Array is already sorted."
- The array is directly used for binary search.

Condition 6: Array is Unsorted

- If arr == sorted(arr) is **False**, the array is unsorted.
- The program prints:
  "Array is unsorted."
- It then uses arr.sort() to sort the array.
- The sorted array is displayed.
- Binary search is then performed on the sorted array.

Error table:

s.no	Error obtained	Rectification
1	Binary search on unsorted array	sort the array first using arr.sort() or provide a sorted array.

WEEK-3

3A) **AIM:** Write a python program to sort the elements using a bubble sort technique. Take an input elements from the user.

Code:

bubble.py - C:/Users/Srivasavi/OneDrive/Documents/CSE25026_DSA/WEEK-3/bubble.py (3.13.14)

File Edit Format Run Options Window Help

```
def bubble(arr):
    for i in range(len(arr)-1):
        for j in range(len(arr)-i-1):
            if arr[j]>arr[j+1]:
                arr[j],arr[j+1]=arr[j+1],arr[j]
    return arr

a=input("Enter Elelments in Array : ")
arr=[int(i) for i in a.split()]
print("Unsorted Array : ",arr)
print("Sorted array : ",bubble(arr))
```

Output:

IDLE Shell 3.13.14

File Edit Shell Debug Options Window Help

```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/Srivasavi/OneDrive/Documents/CSE25026_DSA/WEEK-3/bubble.py
Enter Elelments in Array : 20 40 10 6 7 1 3 2 44 67 79 99
Unsorted Array :  [20, 40, 10, 6, 7, 1, 3, 2, 44, 67, 79, 99]
Sorted array :  [1, 2, 3, 6, 7, 10, 20, 40, 44, 67, 79, 99]
>>>
```

Explanation

Condition 1: arr[j] > arr[j+1] — Elements Need to Be Swapped

- The function bubble_sort(arr) receives the array as an argument.
- The outer for loop runs from 0 to n-1 and controls the number of passes.
- The inner for loop runs from 0 to n-2 and compares adjacent elements.

- The condition arr[j] > arr[j+1] checks whether the current element is greater than the next element.
- If the condition is **True**, the two elements are in the wrong order.
- Therefore, the two elements are swapped using:
- Arr[j],arr[j+1]=arr[j+1],arr[j]
- After the swap, the smaller element moves toward the left and the larger element moves toward the right.

Condition 2: arr[j] <= arr[j+1] — No Swap Required

- If arr[j] > arr[j+1] is **False**, it means the current element is smaller than or equal to the next element.
- Therefore, the elements are already in the correct ascending order.
- No swapping is performed.
- The inner loop continues to compare the next pair of adjacent elements.

Condition 3: for i in range(n) — Repeated Passes

- The outer loop repeats the comparison process for all the elements.
- After each complete pass, the **largest unsorted element moves to the end of the array**.
- The process is repeated until all elements are arranged in ascending order.
- Finally, the function returns the sorted array using:

Error table:

sno	Error obtained	Rectification

1	IndexError: list index out of range	Use correct loop range

3B) **AIM:** Write a python program to sort the elements using a insertion sort technique. Take an input elements from the user.

Code:

insertion.py - C:/Users/Srivasavi/OneDrive/Documents/CSE25026_DSA/WEEK-3/insertion.py (3.13.14)

File Edit Format Run Options Window Help

```
def insertion(arr):
    for i in range(1,len(arr)):
        curr=arr[i]
        j=i-1
        while j>=0 and arr[j]>curr:
            arr[j+1]=arr[j]
            j-=1
        arr[j+1]=curr
    return arr

a=input("Enter Elelments in Array : ")
arr=list(map(int,a.split()))
print("Unsorted Array : ",arr)
print("Sorted array : ",insertion(arr))
```

Output:

IDLE Shell 3.13.14

File Edit Shell Debug Options Window Help

```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/Srivasavi/OneDrive/Documents/CSE25026_DSA/WEEK-3/insertion.py
Enter Elelments in Array : 22 44 77 1 2 11 55 88 900
Unsorted Array :  [22, 44, 77, 1, 2, 11, 55, 88, 900]
Sorted array :  [1, 2, 11, 22, 44, 55, 77, 88, 900]
>>>
```

Explanation:

Condition 1: for i in range(1, len(arr)) — Selecting Element

- The loop starts from the **second element**.
- The first element is considered already sorted.
- i represents the element that needs to be placed correctly.

Condition 2: key = arr[i] — Storing Element

- The current element is stored in key.

```
key = arr[i]
```

- This element will be compared with the elements before it.

Condition 3: j = i - 1 — Getting Previous Position

- j points to the element just before the current element.

```
j = i - 1
```

Condition 4: while j >= 0 and key < arr[j] — Comparing Elements

- The current element is compared with the previous elements.
- If the previous element is greater than key, it is moved one position to the right.

```
while j >= 0 and key < arr[j]:
    arr[j + 1] = arr[j]
    j -= 1
```

- This creates space for the current element.

Condition 5: arr[j + 1] = key — Inserting Element

- The key is placed in its correct position.

```
arr[j + 1] = key
```

- The elements before the current position are now sorted.

Condition 6: return arr — Returning Sorted Array

- After all elements are processed, the function returns the sorted array.

return arr

error table:

sno	Error obtained	Rectification
1	Elements not sorted correctly	Shift elements before insertion

3C) **AIM** : Write a python program to sort the elements using selection sort technique . take an input from the user.

Code:

Selection.py - C:\Users\Srivasavi\OneDrive\Documents\CSE25026_DSA\WEEK-3\Selection.py (3.13.14)

File Edit Format Run Options Window Help

```
def selection(arr):
    for i in range(len(arr)):
        min=i
        for j in range(i,len(arr)):
            if arr[j]<arr[min]:
                min=j
        arr[i],arr[min]=arr[min],arr[i]
    return arr

a=input("Enter Elelments in Array : ")
arr=list(map(int,a.split()))
print("Unsorted Array : ",arr)
print("Sorted array : ",selection(arr))
```

Output:

IDLE Shell 3.13.14

File Edit Shell Debug Options Window Help

```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:\Users\Srivasavi\OneDrive\Documents\CSE25026_DSA\WEEK-3\Selection.py
Enter Elelments in Array : 66 89 91 35 25 70 51
Unsorted Array :  [66, 89, 91, 35, 25, 70, 51]
Sorted array :  [25, 35, 51, 66, 70, 89, 91]
>>>
```

Explanation:

Condition 1: for i in range(n) — Selecting Position

- The loop goes through each position in the array.
- i represents the current position where the smallest element will be placed.

Condition 2: mini = i — Assuming Minimum

- Initially, the element at index i is assumed to be the smallest.

Condition 3: for j in range(i+1,n) — Finding Minimum

- The loop checks all elements after position i.
- It searches for the smallest element.

Condition 4: if arr[mini] > arr[j] — Comparing

- Compares the current minimum with arr[j].
- If arr[j] is smaller, mini is updated.

Condition 5: mini = j — Updating Minimum

- Stores the index of the newly found smallest element.

Condition 6: arr[mini], arr[i] = arr[i], arr[mini] — Swapping

- The smallest element is swapped with the element at position i.
- This places the smallest element in its correct position.

Condition 7: return arr — Returning Result

- Returns the completely sorted array.

Error table:

Sno	Error obtained	Rectification
1	Wrong minimum value	Update min_index properly

3D) **AIM**.: Write a python program to sort the elements using the merge sort technique take an input from the user.

Code:

Merge.py - C:\Users\Srivasavi\OneDrive\Documents\CSE25026_DSA\WEEK-3\Merge.py (3.13.14)

File Edit Format Run Options Window Help

```
def merge(arr):
    if len(arr)<=1:
        return arr
    mid=len(arr)//2
    l=arr[:mid]
    r=arr[mid:]

    merge(l)
    merge(r)

    i=j=k=0
    while i<len(l) and j<len(r):
        if l[i]<=r[j]:
            arr[k]=l[i]
            i+=1
        elif r[j]<l[i]:
            arr[k]=r[j]
            j+=1
        k+=1
    while i<len(l):
        arr[k]=l[i]
        i+=1
        k+=1
    while j<len(r):
        arr[k]=r[j]
        j+=1
        k+=1
    return arr

a=input("Enter Elelments in Array : ")
arr=list(map(int,a.split()))
print("Unsorted Array : ",arr)
print("Sorted array : ",merge(arr))
```

Output:

IDLE Shell 3.13.14

File Edit Shell Debug Options Window Help

```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:\Users\Srivasavi\OneDrive\Documents\CSE25026_DSA\WEEK-3\Merge.py
Enter Elelments in Array : 5 2 7 9 8 1 3 6 4 10
Unsorted Array :  [5, 2, 7, 9, 8, 1, 3, 6, 4, 10]
Sorted array :  [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
>>>
```

Explanation:

Condition 1: if len(arr) <= 1 — Checking the Base Condition

- The function first checks whether the array contains **one or zero elements**.

```
if len(arr) <= 1:
    return arr
```

- If the array has one or zero elements, it is already sorted.
- No further division or comparison is required.
- The function returns the array as it is:

```
return arr
```

Condition 2: mid = len(arr) // 2 — Finding the Middle

- The array is divided into two parts.
- The middle position is calculated using:

```
mid = len(arr) // 2
```

- len(arr) gives the total number of elements.
- // 2 performs integer division and finds the middle index.

Condition 3: left = arr[:mid] — Creating the Left Part

- The elements before the middle position are stored in left.

```
left = arr[:mid]
```

Condition 4: right = arr[mid:] — Creating the Right Part

- The elements from the middle position to the end are stored in right.

```
right = arr[mid:]
```

Condition 5: merge_sort(left) and merge_sort(right) — Sorting Both Parts

- The left and right parts are recursively sorted.

```
merge_sort(left)
merge_sort(right)
```

- The array continues to be divided until each part contains only **one element**.
- A single element is already sorted.
- The smaller sorted parts are then merged together.

Condition 6: i = j = k = 0 — Initializing Indexes

- Three variables are initialized:

i = j = k = 0

- i keeps track of the current element in the **left array**.
- j keeps track of the current element in the **right array**.
- k keeps track of the position in the **original array** where the element will be placed.

Condition 7: while i < len(left) and j < len(right) — Comparing Both Parts

- The program compares elements from the left and right arrays.

while i < len(left) and j < len(right):

- The loop continues as long as both arrays still have elements to compare.
- This allows the program to select the smaller element and place it into the original array.

Condition 8: if left[i] < right[j] — Comparing Elements

- The current elements of the left and right arrays are compared.

if left[i] < right[j]:

- If the left element is smaller than the right element, it is placed into the original array.

arr[k] = left[i]

- Then i is increased:

i += 1

- This moves to the next element of the left array.

Condition 9: else — Taking the Right Element

- If the right element is smaller than or equal to the left element, the right element is placed into the original array.

else:

arr[k] = right[j]

- Then j is increased:

j += 1

- This moves to the next element of the right array.

Condition 10: k += 1 — Moving to the Next Position

- After placing an element into the original array, k is increased.

k += 1

Condition 11: while i < len(left) — Copying Remaining Left Elements

- Sometimes the right array becomes empty before the left array.

while i < len(left):

- Any remaining elements in the left array are copied into the original array.

arr[k] = left[i]

- Then both i and k are increased:

i += 1

k += 1

- The remaining elements are already sorted, so no comparison is needed.

Condition 12: while j < len(right) — Copying Remaining Right Elements

- Sometimes the left array becomes empty before the right array.

while j < len(right):

- Any remaining elements in the right array are copied into the original array.

arr[k] = right[j]

- Then j and k are increased:

j += 1

k += 1

- This completes the merging process.

Error table:

sno	Error obtained	Rectification
1	Recursive error	Use if len(arr) <= 1
2	IndexError	Check left/right array indices

3E) **AIM**: Write a python program to sort the elements using a quick sort technique . take an input elements from the user.

Code:

quick.py - C:/Users/Srivasavi/OneDrive/Documents/CSE25026_DSA/WEEK-3/quick.py (3.13.14)

File Edit Format Run Options Window Help

```
def quick_sort(arr):
    if len(arr) <= 1:
        return arr

    pivot = arr[-1]

    left = []
    right = []

    for x in arr[:-1]:
        if x <= pivot:
            left.append(x)
        else:
            right.append(x)

    return quick_sort(left) + [pivot] + quick_sort(right)

a=input("Enter Elelments in Array : ")
arr=list(map(int,a.split()))
print("Unsorted Array : ",arr)
print("Sorted array : ",quick_sort(arr))
```

Output:

IDLE Shell 3.13.14

File Edit Shell Debug Options Window Help

```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/Srivasavi/OneDrive/Documents/CSE25026_DSA/WEEK-3/quick.py
Enter Elelments in Array : 8 1 3 5 2 10 7 9 4 6
Unsorted Array :  [8, 1, 3, 5, 2, 10, 7, 9, 4, 6]
Sorted array :  [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
>>>
```

Explanation

Condition 1: if arr[j] < pivot — Element Smaller Than Pivot

- The function partition(arr, low, high) is used to divide the array into two parts.
- pivot = arr[high] selects the **last element** of the array as the pivot.
- i = low - 1 is initialized to keep track of the position where elements smaller than the pivot should be placed.
- The for loop checks each element from index low to high-1.

- The condition arr[j] < pivot checks whether the current element is smaller than the pivot.
- If the condition is **True**, i is increased by 1.
- The current element is then swapped with the element at index i:

arr[i], arr[j] = arr[j], arr[i]

- This places all elements smaller than the pivot toward the left side.

Condition 2: arr[j] >= pivot — No Swap Required

- If arr[j] < pivot is **False**, it means the current element is greater than or equal to the pivot.
- No swapping is performed.
- The loop continues to check the next element.
- These elements will remain on the right side of the pivot after partitioning.

Condition 3: arr[i+1], arr[high] = arr[high], arr[i+1] — Placing the Pivot

- After all elements have been checked, the pivot is placed in its correct position.
- The pivot is swapped with the element at index i+1:

arr[i+1], arr[high] = arr[high], arr[i+1]

- After this operation:
  - Elements smaller than the pivot are on the **left**.
  - The pivot is in its **correct position**.
  - Elements greater than or equal to the pivot are on the **right**.
- The function returns the pivot position:

return i+1

Condition 4: if low < high — Continue Quick Sort

- The function quick_sort(arr, low, high) is used to sort the array.
- The condition low < high checks whether the current part of the array contains more than one element.

- If the condition is **True**, the array needs to be sorted.
- The partition() function is called:

pi = partition(arr, low, high)

- pi stores the final position of the pivot.

Condition 5: quick_sort(arr, low, pi-1) — Sort Left Part

- After partitioning, the elements on the left side of the pivot are sorted separately.
- This is done using:

quick_sort(arr, low, pi-1)

- The pivot itself is not included because it is already in its correct position.

Condition 6: quick_sort(arr, pi+1, high) — Sort Right Part

- The elements on the right side of the pivot are sorted separately.
- This is done using:

quick_sort(arr, pi+1, high)

- Again, the pivot is not included because it is already correctly positioned.

Input Section

- n = int(input("enter the no of elements:")) takes the number of elements from the user.
- arr = [] creates an empty list to store the elements.
- The for loop runs n times and takes each element from the user:
- for i in range(n):

arr.append(int(input("enter the elements:")))

- Each entered value is converted into an integer and added to the array using append().

Calling the Quick Sort Function

quick_sort(arr, 0, n-1)

- This calls the quick_sort() function.
- 0 represents the first index of the array.
- n-1 represents the last index of the array.
- The complete array is therefore passed for sorting.

.

Error table:

sno	Error obtained	Rectification
1	IndexError	Check i, j, and pivot boundaries
2	Wrong swapping conditions	By shifting to the correct spaces

WEEK-4

4A) **AIM:** Write a python program to perform linked list operations by taking the input from the user

Code:

Single_LL.py - C:/Users/Srivasavi/OneDrive/Documents/CSE25026_DSA/WEEK-4/Single_LL.py (3.13.14)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self,data):
        self.data=data
        self.next=None

class LinkedList:
    def __init__(self):
        self.head=None

    def insert_at_beginning(self,data):
        n=Node(data)
        if self.head is None:
            self.head=n
            return
        n.next=self.head
        self.head=n

    def insert_at_end(self,data):
        n=Node(data)
        temp=self.head
        while temp.next:
            temp=temp.next
        temp.next=n

    def insert_at_index(self,data,index):
        if self.head is None:
            print("No data Available")
            return
        new=Node(data)
        temp=self.head
        if index == 0:
            new.next = self.head
            self.head = new
            return
        for i in range(index-1):
            temp=temp.next
        new.next=temp.next
        temp.next=new

    def delete_at_beginning(self):
        if self.head is None:
            print("No data Available")
            return
        temp=self.head
        self.head=temp.next
```

```
    def delete_at_end(self):
        if not self.had:
            print("No data Available")
            return
        temp=self.head
        while temp.next:
            temp=temp.next
        temp.next=None

    def remove_with_value(self,target):
        if self.head is None:
            print("No data Available")
            return
        else:
            temp=self.head
            if temp and temp.data == target:
                self.head=temp.next
                return
            while temp.next and temp.next.data != target:
                temp=temp.next
            if temp.next:
                temp.next=temp.next.next
            else:
                print("No Value Present")

    def count(self):
        temp=self.head
        count=0
        while temp:
            count+=1
            temp=temp.next
        print(count)

    def display(self):
        if self.head is None:
            print("No data available")
            return
        temp=self.head
        while temp:
            print(temp.data)
            temp=temp.next
```

```
l=LinkedList()

while True:
    print("\n----- Double Linked List MENU -----")
    print("1. Insert at the Beginning")
    print("2. Insert at the end")
    print("3. Insert at Index")
    print("4. Delete at Beginning")
    print("5. Delete at End")
    print("6. Delete with Value")
    print("7. Count")
    print("8. Display")
    print("9. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        data = int(input("Enter the element: "))
        l.insert_at_beginning(data)

    elif choice == 2:
        l.insert_at_end()

    elif choice == 3:
        l.display_from_head()

    elif choice == 4:
        l.delete_at_beginning()

    elif choice == 5:
        l.delete_at_end()

    elif choice == 6:
        data = int(input("Enter the element: "))
        l.remove_with_value(data)

    elif choice == 8:
        l.display_from_tail()

    elif choice == 7:
        l.count()

    elif choice == 9:
        print("Exiting program...")
        break
```

Output:

```
>>>
= RESTART: C:\Users\CSE-Lab1\Desktop\DSA_CSE25026\WEEK-4\SingleLinkedLists.py

----- Single Linked List MENU -----
1. Insert at the Beginning
2. Insert at the end
3. Insert at Index
4. Delete at Beginning
5. Delete at End
6. Delete with Value
7. Count
8. Display
9. Exit
Enter your choice: 1
Enter the element: 121

----- Single Linked List MENU -----
1. Insert at the Beginning
2. Insert at the end
3. Insert at Index
4. Delete at Beginning
5. Delete at End
6. Delete with Value
7. Count
8. Display
9. Exit
Enter your choice: 2
Enter the element: 121

----- Single Linked List MENU -----
1. Insert at the Beginning
2. Insert at the end
3. Insert at Index
4. Delete at Beginning
5. Delete at End
6. Delete with Value
7. Count
8. Display
9. Exit
Enter your choice: 3
Enter the element: 121
Enter the Index of the element : 1
```

```
----- Single Linked List MENU -----
1. Insert at the Beginning
2. Insert at the end
3. Insert at Index
4. Delete at Beginning
5. Delete at End
6. Delete with Value
7. Count
8. Display
9. Exit
Enter your choice: 3
Enter the element: 121
Enter the Index of the element : 1

----- Single Linked List MENU -----
1. Insert at the Beginning
2. Insert at the end
3. Insert at Index
4. Delete at Beginning
5. Delete at End
6. Delete with Value
7. Count
8. Display
9. Exit
Enter your choice: 8
121
121
121

----- Single Linked List MENU -----
1. Insert at the Beginning
2. Insert at the end
3. Insert at Index
4. Delete at Beginning
5. Delete at End
6. Delete with Value
7. Count
8. Display
9. Exit
Enter your choice: 9
Exiting program...
>>>
```

Explanation:

1. A **singly linked list** is a collection of nodes where each node contains **two parts**: data and a link to the next node.
2. Each node contains:
   - **Data** – stores the value.
   - **Next (next)** – stores the reference/address of the next node.
3. The program maintains two references:
   - **Head** – points to the first node.
   - **Tail** – points to the last node.
4. Initially, when the list is empty, both head and tail are set to None.

5. A **Node class** is used to create individual nodes and store the data and reference to the next node.
6. During **creation of the linked list**, the user enters the number of nodes and their data. The nodes are created one by one and added to the list.
7. For the first node, both **head and tail** point to the same node, and the next of the tail is None.
8. During **insertion at the beginning**, a new node is created and placed before the current head. The new node's next points to the current head, and the new node becomes the new head.
9. During **insertion at the end**, the new node is added after the current tail. The current tail's next points to the new node, and the new node becomes the new tail.
10. During **insertion at a specific index**, the list is traversed until the required position is reached. The new node is inserted between the appropriate nodes by adjusting their next links.
11. During **deletion by value**, the list is searched for the node containing the given value. The node is removed by making the previous node point to the node after the deleted node.
12. If the node to be deleted is the **head**, the next node becomes the new head.
13. If the node to be deleted is the **tail**, the previous node becomes the new tail and its next is set to None.
14. If the list contains only one node and that node is deleted, both **head and tail are set to None**.
15. During **deletion at the beginning**, the head node is removed and the next node becomes the new head.
16. During **deletion at the end**, the list is traversed to find the node before the tail. That node becomes the new tail and its next is set to None.
17. The **count operation** traverses the list from the head to the end and counts the total number of nodes.
18. The **display/traverse operation** starts from the head and moves through the list using the next link until None is reached.
20. The program continues to execute until the user selects the **Exit** option.

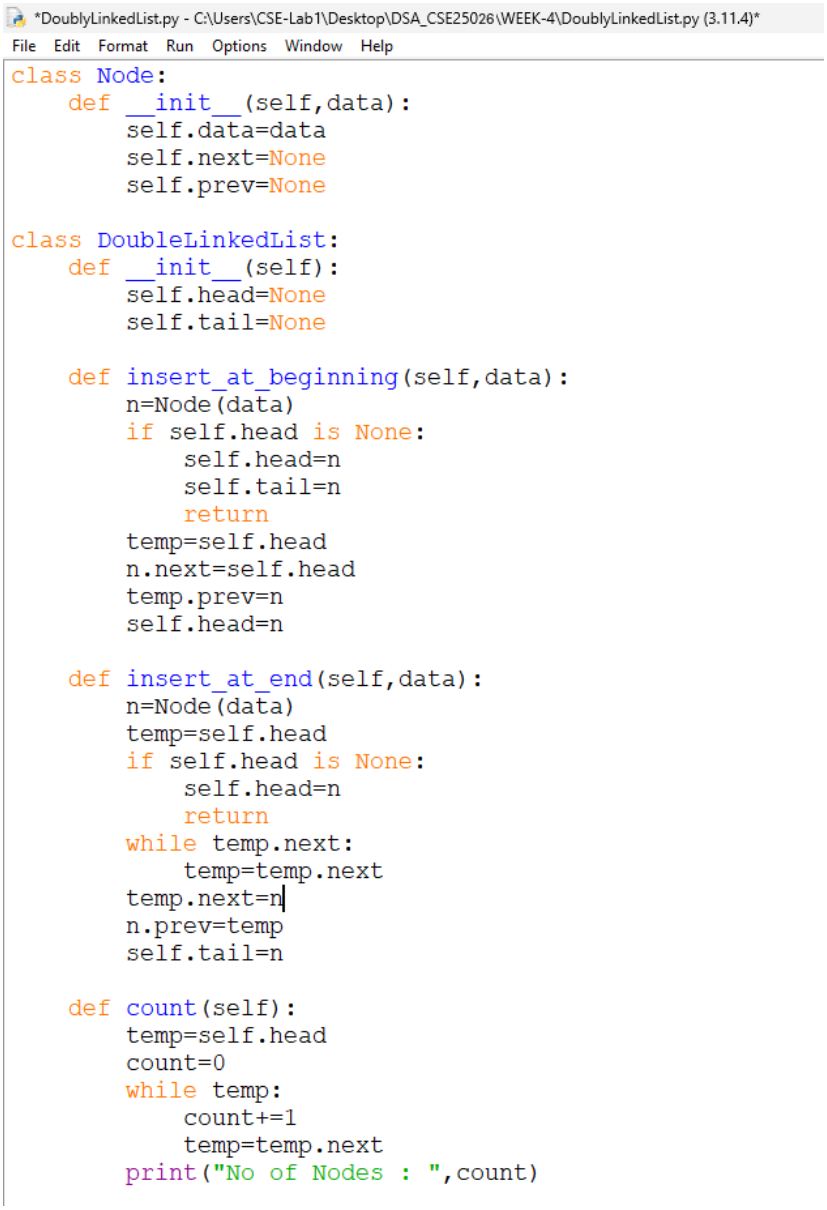
Error table:

sno	Error obtained	rectification

1	Infinite loop or incomplete traversal due to not updating current	Move to the next node using temp = temp.next
2	Forgetting to update head while inserting/deleting at the beginning	Update head to point to the new first node using head = new_node or head = head.next

4B) **AIM:** Write a python program to perform double linked list operations by taking input from the user .

Code:



```
    def display_from_tail(self):
        if self.head is None:
            print("No data available")
            return
        temp=self.tail
        while temp:
            print(temp.data)
            temp=temp.prev


d=DoubleLinkedList()

while True:
    print("\n----- Double Linked List MENU -----")
    print("1. Insert at the Beginning")
    print("2. Insert at the end")
    print("3. Display from Head")
    print("4. Display from Tail")
    print("5. Count")
    print("6. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        data = int(input("Enter the element: "))
        d.insert_at_beginning(data)

    elif choice == 2:
        d.insert_at_end()

    elif choice == 3:
        d.display_from_head()

    elif choice == 4:
        d.display_from_tail()

    elif choice == 5:
        d.count()

    elif choice == 6:
        print("Exiting program...")
        break

    else:
        print("Invalid choice.")
```

Output:

```
>>>
= RESTART: C:\Users\CSE-Lab1\Desktop\DSA_CSE25026\WEEK-4\DoublyLinkedList.py

----- Double Linked List MENU -----
1. Insert at the Beginning
2. Insert at the end
3. Display from Head
4. Display from Tail
5. Count
6. Exit
Enter your choice: 1
Enter the element: 9

----- Double Linked List MENU -----
1. Insert at the Beginning
2. Insert at the end
3. Display from Head
4. Display from Tail
5. Count
6. Exit
Enter your choice: 2
Enter the element: 99

----- Double Linked List MENU -----
1. Insert at the Beginning
2. Insert at the end
3. Display from Head
4. Display from Tail
5. Count
6. Exit
Enter your choice: 5
No of Nodes :  2

----- Double Linked List MENU -----
1. Insert at the Beginning
2. Insert at the end
3. Display from Head
4. Display from Tail
5. Count
6. Exit
Enter your choice: 6
Exiting program...
>>>
```

Explanation:

1. A **doubly linked list** is a collection of nodes where each node contains **three parts**: data, a link to the previous node, and a link to the next node.
2. Each node has:
   - **Data** – stores the value.
   - **Previous (prev)** – points to the previous node.
   - **Next (next)** – points to the next node.
3. The program maintains two references:

- **Head** – points to the first node.
- **Tail** – points to the last node.

4. Initially, when the list is empty, both head and tail are set to None.
5. A **Node class** is used to create individual nodes and maintain the data, prev, and next references.
6. During **creation of the linked list**, the user enters the number of nodes and their data. The nodes are added one by one at the end of the list.
7. For the first node, both **head and tail** point to the same node. The prev of the head and the next of the tail are None.
8. During **insertion at the beginning**, a new node is created and placed before the current head. The new node becomes the head, its next points to the previous head, and the previous head's prev points back to the new node.
9. During **insertion at the end**, the new node is placed after the current tail. The new node's prev points to the old tail, the old tail's next points to the new node, and the new node becomes the tail.
10. During **insertion at a specific index**, the list is traversed until the required position is reached. The new node is inserted between the previous and next nodes, and both prev and next links are updated.
11. During **deletion by value**, the list is searched for the node containing the given value. Once found, its previous and next nodes are connected to each other, removing the selected node from the list.
12. If the node to be deleted is the **head**, the next node becomes the new head and its prev is set to None.
13. If the node to be deleted is the **tail**, the previous node becomes the new tail and its next is set to None.
14. If the list contains only one node and that node is deleted, both **head and tail are set to None**.
15. During **deletion at the beginning**, the head node is removed and the next node becomes the new head. The new head's prev is set to None.

16. During **deletion at the end**, the tail node is removed and the previous node becomes the new tail. The new tail's next is set to None.
17. The **count operation** traverses the list from the head to the tail and counts the total number of nodes.
18. The **display/traverse operation** starts from the head and moves forward using the next link to display all the nodes.
19. Since it is a doubly linked list, the program can also traverse the list **backwards**, starting from the tail and using the prev links.
20. The program displays the **head and tail** values to show the starting and ending nodes of the list.
21. The important relationships maintained in a doubly linked list are:

head.prev = None

tail.next = None

22. The overall structure of a doubly linked list is:

None ← [10] ⇄ [20] ⇄ [30] ⇄ [40] → None

↑ ↑

HEAD TAIL

23. The arrows in both directions allow the list to be traversed **forward as well as backward**.
24. The program is **menu-driven**, allowing the user to perform creation, insertion, deletion, counting, displaying, and other operations repeatedly until the **Exit** option is selected.

Error table:

Sno	Error obtained	Rectification
1	Forgetting to update the prev pointer	Update both prev and next pointers whenever a node is inserted
2		Maintain head.prev = None

	Incorrect head.prev	

4c)**AIM:** write a python program to implement a circular linked list which could perform the below operations by taking an input from the user

I .create a linked list

ii. insert at beginning

iii. insert at end

iv. insert at specific index

v. delete by value

vi. delete at beginning

vii. delete at the end

viii. count no of nodes

ix. display or travsere

x: display the head and tail

xi. exit

Code:

circular.py - C:/Users/Srivasavi/OneDrive/Documents/CSE25026_DSA/WEEK-4/circular.py (3.13.14)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class CircularLinkedList:
    def __init__(self):
        self.head = None
        self.tail = None

    def create(self):
        n = int(input("Enter number of nodes: "))
        for i in range(n):
            data = int(input("Enter value: "))
            self.insert_end(data)
        print("Linked list created.")

    def insert_begin(self, data):
        new = Node(data)
        if self.head is None:
            self.head = self.tail = new
            new.next = new
        else:
            new.next = self.head
            self.head = new
            self.tail.next = self.head

    def insert_end(self, data):
        new = Node(data)
        if self.head is None:
            self.head = self.tail = new
            new.next = new
        else:
            self.tail.next = new
            self.tail = new
            self.tail.next = self.head

    def insert_index(self, data, index):
        if index < 0:
            print("Invalid index.")
            return

        if index == 0:
            self.insert_begin(data)
            return
```

```
        new = Node(data)
        new.next = temp.next
        temp.next = new

    def delete_value(self, key):
        if self.head is None:
            print("List is empty.")
            return

        curr = self.head
        prev = self.tail

        while True:
            if curr.data == key:
                if curr == self.head:
                    self.delete_begin()
                elif curr == self.tail:
                    self.delete_end()
                else:
                    prev.next = curr.next
                    print("Node deleted.")
                return

            prev = curr
            curr = curr.next

            if curr == self.head:
                break

        print("Value not found.")

    def delete_begin(self):
        if self.head is None:
            print("List is empty.")
            return

        if self.head == self.tail:
            self.head = self.tail = None
        else:
            self.head = self.head.next
            self.tail.next = self.head

        print("Beginning node deleted.")
```

```
def delete_end(self):
    if self.head is None:
        print("List is empty.")
        return

    if self.head == self.tail:
        self.head = self.tail = None
    else:
        temp = self.head
        while temp.next != self.tail:
            temp = temp.next

        self.tail = temp
        self.tail.next = self.head

    print("Last node deleted.")

def count(self):
    if self.head is None:
        return 0

    count = 1
    temp = self.head

    while temp.next != self.head:
        count += 1
        temp = temp.next

    return count

def display(self):
    if self.head is None:
        print("List is empty.")
        return

    temp = self.head
    while True:
        print(temp.data, end=" -> ")
        temp = temp.next
        if temp == self.head:
            break
```

```
def delete_begin(self):
    if self.head is None:
        print("List is empty.")
        return

    if self.head == self.tail:
        self.head = self.tail = None
    else:
        self.head = self.head.next
        self.tail.next = self.head

    print("Beginning node deleted.")

def delete_end(self):
    if self.head is None:
        print("List is empty.")
        return

    if self.head == self.tail:
        self.head = self.tail = None
    else:
        temp = self.head
        while temp.next != self.tail:
            temp = temp.next

        self.tail = temp
        self.tail.next = self.head

    print("Last node deleted.")

def count(self):
    if self.head is None:
        return 0

    count = 1
    temp = self.head

    while temp.next != self.head:
        count += 1
        temp = temp.next

    return count
```

```
    def display(self):
        if self.head is None:
            print("List is empty.")
            return

        temp = self.head
        while True:
            print(temp.data, end=" -> ")
            temp = temp.next
            if temp == self.head:
                break

        print("(Head)")

    def show_head_tail(self):
        if self.head is None:
            print("List is empty.")
        else:
            print("Head:", self.head.data)
            print("Tail:", self.tail.data)


cll = CircularLinkedList()

while True:
    print("\n----- CIRCULAR LINKED LIST -----")
    print("1. Create Linked List")
    print("2. Insert at Beginning")
    print("3. Insert at End")
    print("4. Insert at Specific Index")
    print("5. Delete by Value")
    print("6. Delete at Beginning")
    print("7. Delete at End")
    print("8. Count Nodes")
    print("9. Display / Traverse")
    print("10. Display Head and Tail")
    print("11. Exit")
```

```
        elif ch == 2:
            data = int(input("Enter value: "))
            cll.insert_begin(data)

        elif ch == 3:
            data = int(input("Enter value: "))
            cll.insert_end(data)

        elif ch == 4:
            data = int(input("Enter value: "))
            index = int(input("Enter index: "))
            cll.insert_index(data, index)

        elif ch == 5:
            key = int(input("Enter value to delete: "))
            cll.delete_value(key)

        elif ch == 6:
            cll.delete_begin()

        elif ch == 7:
            cll.delete_end()

        elif ch == 8:
            print("Number of nodes:", cll.count())

        elif ch == 9:
            cll.display()

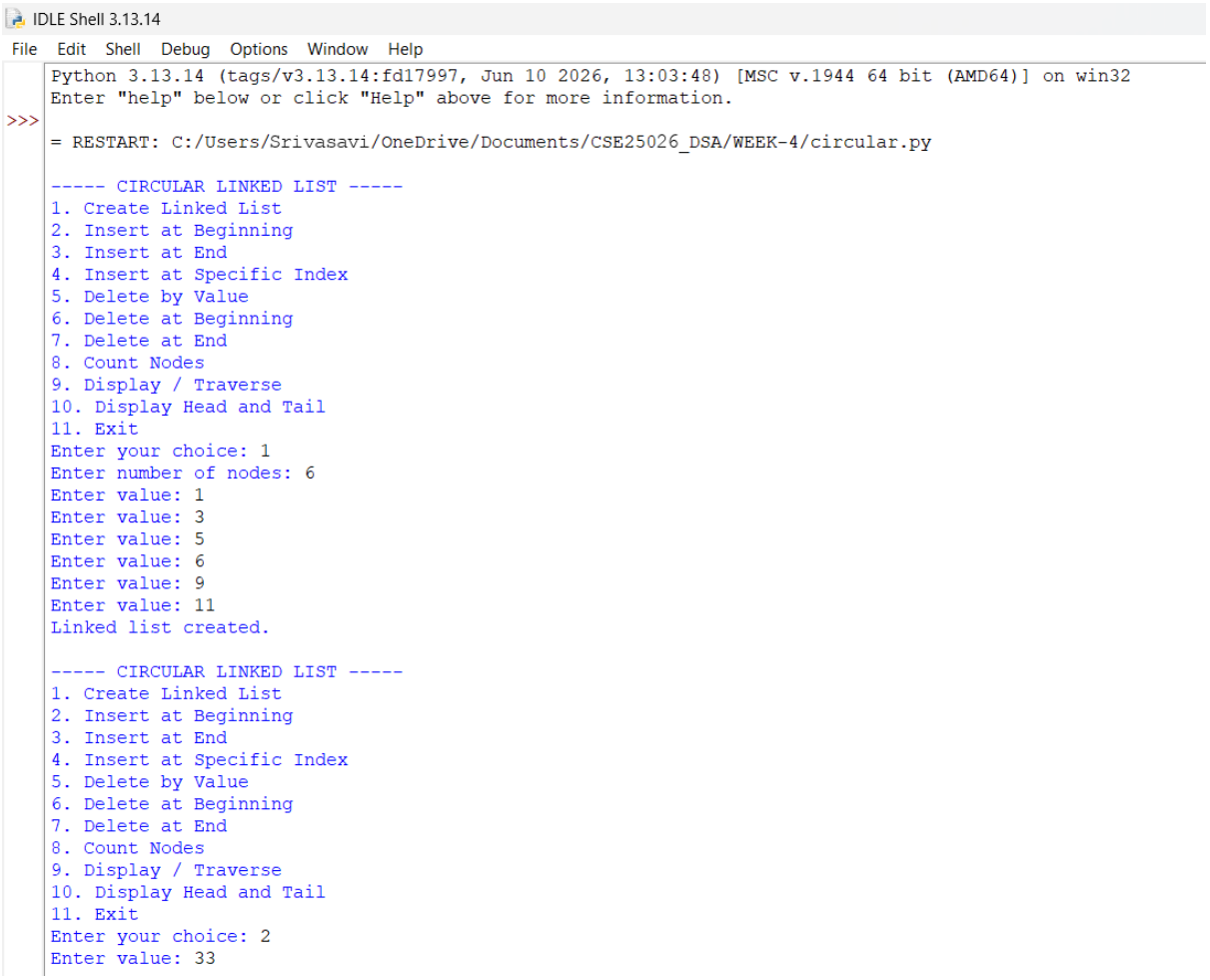
        elif ch == 10:
            cll.show_head_tail()

        elif ch == 11:
            print("Exiting program.")
            break

        else:
            print("Invalid choice!")

    except ValueError:
        print("Please enter a valid integer.")
```

Output:


IDLE Shell 3.13.14
File Edit Shell Debug Options Window Help
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/Srivasavi/OneDrive/Documents/CSE25026_DSA/WEEK-4/circular.py
----- CIRCULAR LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete at Beginning
7. Delete at End
8. Count Nodes
9. Display / Traverse
10. Display Head and Tail
11. Exit
Enter your choice: 1
Enter number of nodes: 6
Enter value: 1
Enter value: 3
Enter value: 5
Enter value: 6
Enter value: 9
Enter value: 11
Linked list created.
----- CIRCULAR LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete at Beginning
7. Delete at End
8. Count Nodes
9. Display / Traverse
10. Display Head and Tail
11. Exit
Enter your choice: 2
Enter value: 33

```
----- CIRCULAR LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete at Beginning
7. Delete at End
8. Count Nodes
9. Display / Traverse
10. Display Head and Tail
11. Exit
Enter your choice: 4
Enter value: 56
Enter index: 5

----- CIRCULAR LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete at Beginning
7. Delete at End
8. Count Nodes
9. Display / Traverse
10. Display Head and Tail
11. Exit
Enter your choice: 7
Last node deleted.

----- CIRCULAR LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete at Beginning
7. Delete at End
8. Count Nodes
9. Display / Traverse
10. Display Head and Tail
11. Exit
Enter your choice: 6
Beginning node deleted.
```

```
----- CIRCULAR LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete at Beginning
7. Delete at End
8. Count Nodes
9. Display / Traverse
10. Display Head and Tail
11. Exit
Enter your choice: 8
Number of nodes: 6

----- CIRCULAR LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete at Beginning
7. Delete at End
8. Count Nodes
9. Display / Traverse
10. Display Head and Tail
11. Exit
Enter your choice: 9
1 -> 3 -> 5 -> 6 -> 56 -> 9 -> (Head)

----- CIRCULAR LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete at Beginning
7. Delete at End
8. Count Nodes
9. Display / Traverse
10. Display Head and Tail
11. Exit
Enter your choice: 10
Head: 1
```

```
----- CIRCULAR LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete at Beginning
7. Delete at End
8. Count Nodes
9. Display / Traverse
10. Display Head and Tail
11. Exit
Enter your choice: 11
Exiting program.
>>>
```

Explanation :

1. A **circular singly linked list** is a collection of nodes where each node contains **data** and a **link to the next node**.
2. In a circular linked list, the **last node points back to the first node (head)** instead of pointing to None.
3. The program uses two references:
   - **Head** – points to the first node.
   - **Tail** – points to the last node.
4. Initially, when the list is empty, both head and tail are set to None.
5. A **Node class** is used to create individual nodes and store the data and reference to the next node.
6. During **creation of the linked list**, the user enters the number of nodes and their data. Each new node is inserted at the end of the list.
7. If the list is empty, the first node becomes both the **head and tail**, and its next points back to itself.
8. During **insertion at the beginning**, the new node is placed before the current head. The new node becomes the head, and the tail is connected to the new head.
9. During **insertion at the end**, the new node is added after the current tail. The new node becomes the tail, and its next points to the head.

10. During **insertion at a specific index**, the list is traversed until the required position is reached, and the new node is inserted between two existing nodes.
11. During **deletion by value**, the list is searched for the node containing the given value. The node is removed by connecting its previous node directly to its next node.
12. If the node to be deleted is the **head**, the next node becomes the new head and the tail is connected to it.
13. If the node to be deleted is the **tail**, the previous node becomes the new tail and its next is connected to the head.
14. During **deletion at the beginning**, the head node is removed and the next node becomes the new head.
15. During **deletion at the end**, the node before the tail is found and made the new tail.
16. The **count operation** traverses the list and counts all the nodes. Traversal stops when the head is reached again.
17. The **display/traverse operation** starts from the head and visits every node until it reaches the head again.
18. Since the list is circular, traversal does not use current != None. Instead, it stops when current becomes equal to head.
19. The program also displays the **head, tail, and tail.next** to show the circular connection.
20. The program is **menu-driven**, so the user can repeatedly perform different operations until the **Exit** option is selected.
21. The most important property maintained throughout the program is:

tail.next = head

22. Thus, the structure of the circular linked list is:

HEAD → Node → Node → Node → TAIL

↑ |

|_____|

This ensures that the linked list remains **circular after every insertion and deletion operation**.

Error table:

sno	Error obtained	Rectification
1	Using temp != None as the traversal condition	Stop traversal when temp == head
2	Forgetting to update tail.next when the head changes	After changing the head, update tail.next = head

WEEK-5

5A) **AIM:** Write a python program to implement the stack by using arrays.

Code:

Stack_List.py - C:/Users/Srivasavi/OneDrive/Documents/CSE25026_DSA/Stack_List.py (3.13.14)

File Edit Format Run Options Window Help

```
class Stack_Array:
    def __init__(self, length):
        self.length = length
        self.stack = []
        self.top = -1

    def stack_push(self, data):
        if len(self.stack) == self.length:
            print("Stack is full")
            return

        self.stack.append(data)
        self.top += 1
        print("Element pushed successfully")

    def stack_pop(self):
        if len(self.stack) == 0:
            print("Stack is empty")
            return

        print(self.stack.pop(), "removed")
        self.top -= 1

    def peek(self):
        if len(self.stack) == 0:
            print("Stack is empty")
        else:
            print("Top element:", self.stack[-1])

    def top_index(self):
        if len(self.stack) == 0:
            print("Stack is empty")
        else:
            print("Top element index:", self.top)

    def display(self):
        if len(self.stack) == 0:
            print("Stack is empty")
        else:
            print("Stack elements:")
            for i in range(len(self.stack) - 1, -1, -1):
                print(self.stack[i])

    def count(self):
        print("Number of elements:", len(self.stack))
```

```
s = Stack_Array(5)

while True:
    print("\n----- STACK MENU -----")
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Count")
    print("6. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        data = int(input("Enter the element: "))
        s.stack_push(data)

    elif choice == 2:
        s.stack_pop()

    elif choice == 3:
        s.peek()

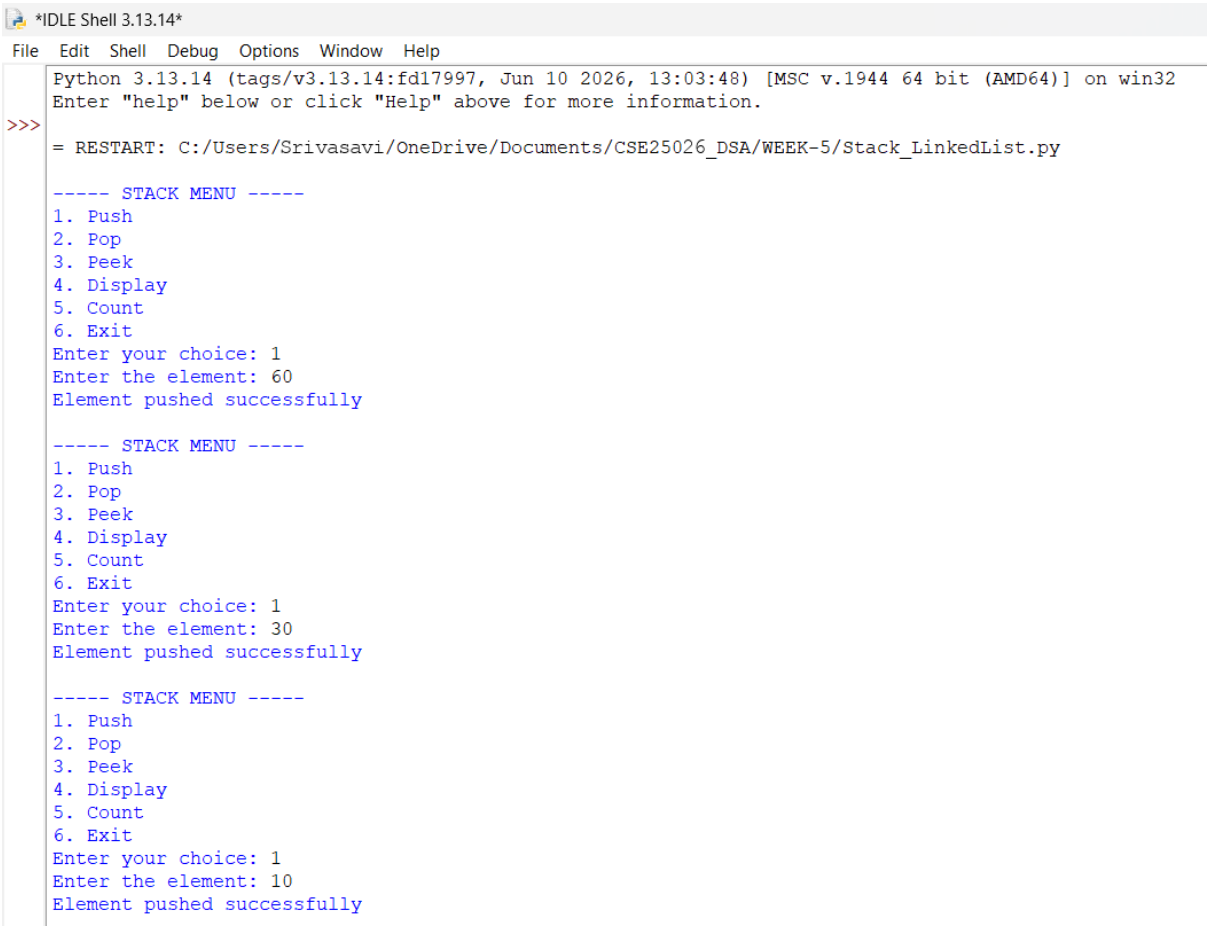
    elif choice == 4:
        s.display()

    elif choice == 5:
        s.count()

    elif choice == 6:
        print("Exiting program...")
        break

    else:
        print("Invalid choice.")
```

Output:


IDLE Shell 3.13.14
File Edit Shell Debug Options Window Help
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/Srivasavi/OneDrive/Documents/CSE25026_DSA/WEEK-5/Stack_LinkedList.py
----- STACK MENU -----
1. Push
2. Pop
3. Peek
4. Display
5. Count
6. Exit
Enter your choice: 1
Enter the element: 60
Element pushed successfully
----- STACK MENU -----
1. Push
2. Pop
3. Peek
4. Display
5. Count
6. Exit
Enter your choice: 1
Enter the element: 30
Element pushed successfully
----- STACK MENU -----
1. Push
2. Pop
3. Peek
4. Display
5. Count
6. Exit
Enter your choice: 1
Enter the element: 10
Element pushed successfully

```
----- STACK MENU -----
1. Push
2. Pop
3. Peek
4. Display
5. Count
6. Exit
Enter your choice: 1
Enter the element: 10
Element pushed successfully

----- STACK MENU -----
1. Push
2. Pop
3. Peek
4. Display
5. Count
6. Exit
Enter your choice: 4
Stack elements:
10
30
60

----- STACK MENU -----
1. Push
2. Pop
3. Peek
4. Display
5. Count
6. Exit
Enter your choice: 5
Number of elements: 3

----- STACK MENU -----
1. Push
2. Pop
3. Peek
4. Display
5. Count
6. Exit
Enter your choice: 6
Exiting program...
```

Explanation:

1. Class and Initialization

- A class named Stack is created.
- The __init__() method acts as the constructor.
- An empty list self.stack is created to store stack elements.
- The stack follows the **LIFO (Last In, First Out)** principle.

2. Push Operation

- The push() method is used to insert an element into the stack.
- The append() function adds the element to the end of the list.
- The newly inserted element becomes the **top element**.
- A message is displayed after the element is inserted.

3. Pop Operation

- The pop() method is used to remove the top element.
- The program first checks whether the stack is empty.
- If the stack is empty, **Stack Underflow** is displayed.
- Otherwise, the pop() function removes the last element.
- The removed element is displayed.

4. Peek Operation

- The peek() method is used to view the top element.
- The program checks whether the stack is empty.
- If the stack is empty, **Stack is empty** is displayed.
- Otherwise, self.stack[-1] is used to access the top element.
- The top element is displayed without removing it.

5. Display Operation

- The display() method is used to display all stack elements.
- The program first checks whether the stack is empty.
- If empty, **Stack is empty** is displayed.
- Otherwise, a for loop is used to traverse the stack.
- The elements are displayed from **top to bottom**.

6. Count Operation

- The count() method is used to find the number of elements in the stack.
- The len() function returns the number of elements.

- The count is displayed to the user.

Error table:

sno	Error obtained	Error rectification
1	Stack underflow	Check whether the stack is empty before poping
2	Name error	Define the class before creating the object

5B) **AIM:** Write a python program to implement stack using the linked list.

Code:

Stack_LinkedList.py - C:\Users\Srivasavi\OneDrive\Documents\CSE25026_DSA\WEEK-5\Stack_LinkedList.py (3.13.14)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class Stack_LinkedList:
    def __init__(self):
        self.top = None

    def stack_push(self, data):
        new = Node(data)

        new.next = self.top
        self.top = new

        print("Element pushed successfully")

    def stack_pop(self):
        if self.top is None:
            print("Stack is Empty")
        else:
            print(self.top.data, "removed")
            self.top = self.top.next

    def peek(self):
        if self.top is None:
            print("Stack is empty")
        else:
            print("Top element:", self.top.data)

    def display(self):
        if self.top is None:
            print("Stack is empty")
        else:
            print("Stack elements:")

            temp = self.top

            while temp:
                print(temp.data)
                temp = temp.next
```

```
    def count(self):
        if self.top is None:
            print("Number of elements: 0")
        else:
            temp = self.top
            count = 0

            while temp:
                count += 1
                temp = temp.next

            print("Number of elements:", count)

s = Stack_LinkedList()

while True:
    print("\n----- STACK MENU -----")
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Count")
    print("6. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        data = int(input("Enter the element: "))
        s.stack_push(data)

    elif choice == 2:
        s.stack_pop()

    elif choice == 3:
        s.peek()

    elif choice == 4:
        s.display()

    elif choice == 5:
        s.count()

    elif choice == 6:
        print("Exiting program...")
        break

    else:
        print("Invalid choice.")
```

Output:

```
>>>
= RESTART: C:\Users\Srivasavi\OneDrive\Documents\CSE25026_DSA\WEEK-5\Stack_LinkedList.py

----- STACK MENU -----
1. Push
2. Pop
3. Peek
4. Display
5. Count
6. Exit
Enter your choice: 1
Enter the element: 11
Element pushed successfully

----- STACK MENU -----
1. Push
2. Pop
3. Peek
4. Display
5. Count
6. Exit
Enter your choice: 1
Enter the element: 22
Element pushed successfully

----- STACK MENU -----
1. Push
2. Pop
3. Peek
4. Display
5. Count
6. Exit
Enter your choice: 4
Stack elements:
22
11
```

```
----- STACK MENU -----
1. Push
2. Pop
3. Peek
4. Display
5. Count
6. Exit
Enter your choice: 4
Stack elements:
22
11

----- STACK MENU -----
1. Push
2. Pop
3. Peek
4. Display
5. Count
6. Exit
Enter your choice: 5
Number of elements: 2

----- STACK MENU -----
1. Push
2. Pop
3. Peek
4. Display
5. Count
6. Exit
Enter your choice: 6
Exiting program...
>>>
```

Explanation:

1. Node Class

- A class named Node is created to represent each element of the linked list.
- The __init__() method initializes a node.
- self.data stores the value of the node.
- self.next stores the reference to the next node.
- Initially, self.next is set to None.

2. Stack Class

- A class named Stack is created to implement the stack operations.
- The __init__() method initializes the stack.

- self.top = None indicates that the stack is initially empty.
- The top variable stores the reference to the top node of the stack.

3. Push Operation

- The push() method is used to insert an element into the stack.
- A new node is created using Node(data).
- The new node is connected to the current top node using:

new_node.next = self.top

- The new node is then made the top node using:

self.top = new_node

- The inserted element is displayed.
- The operation follows the **LIFO (Last In, First Out)** principle.

4. Pop Operation

- The pop() method is used to remove the top element.
- The program first checks whether self.top is None.
- If self.top is None, the stack is empty and **Stack Underflow** is displayed.
- Otherwise, the data of the top node is stored in data.
- The top is moved to the next node using:

self.top = self.top.next

- The removed element is displayed.

5. Peek Operation

- The peek() method is used to view the top element without removing it.
- The program checks whether the stack is empty.
- If the stack is empty, **Stack is empty** is displayed.
- Otherwise, self.top.data is used to display the top element.
- The stack remains unchanged after the peek operation.

6. Display Operation

- The display() method is used to display all the elements of the stack.
- The program first checks whether the stack is empty.
- A temporary variable current is initialized with self.top.
- A while loop is used to traverse the linked list.
- current.data displays the value of each node.
- current = current.next moves to the next node.
- The elements are displayed from **top to bottom**.

7. Count Operation

- The count() method is used to find the number of elements in the stack.
- A variable count is initialized to 0.
- A temporary variable current starts from self.top.
- The linked list is traversed using a while loop.
- The count value is increased for every node.
- The total number of elements is displayed.

Error table:

sno	Error obtained	Error rectification
1	Incorrect push	Set new_node.next = self.top before updating self.top
2	Infinte loop	Use break when choice == 6

WEEK-6

6a) **Aim**: write a python program to implement queue by using the arrays

Code:

QUEUE_LIST.py - C:/Users/Srivasavi/OneDrive/Documents/CSE25026_DSA/QUEUE_LIST.py (3.13.14)

File Edit Format Run Options Window Help

```
class Queue:
    def __init__(self):
        self.queue = []
        self.front = -1
        self.rear = -1

    # Enqueue operation
    def enqueue(self, data):
        self.queue.append(data)
        self.rear += 1

        if self.front == -1:
            self.front = 0

        print(data, "inserted into the queue.")

    # Dequeue operation
    def dequeue(self):
        if self.front == -1 or self.front > self.rear:
            print("Queue Underflow. Queue is empty.")
        else:
            data = self.queue[self.front]
            self.front += 1
            print(data, "deleted from the queue.")

    # Peek operation
    def peek(self):
        if self.front == -1 or self.front > self.rear:
            print("Queue is empty.")
        else:
            print("Front element is:", self.queue[self.front])

    # Display all elements
    def display(self):
        if self.front == -1 or self.front > self.rear:
            print("Queue is empty.")
        else:
            print("Queue elements are:")
            for i in range(self.front, self.rear + 1):
                print(self.queue[i])
```

QUEUE_LIST.py - C:/Users/Srivasavi/OneDrive/Documents/CSE25026_DSA/QUEUE_LIST.py (3.13.14)

File Edit Format Run Options Window Help

```
                print(self.queue[i])

    # Count number of elements
    def count(self):
        if self.front == -1 or self.front > self.rear:
            print("Number of elements: 0")
        else:
            print("Number of elements:", self.rear - self.front + 1)


# Main program
q = Queue()

while True:
    print("\n--- QUEUE MENU ---")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Count")
    print("6. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        data = int(input("Enter the element: "))
        q.enqueue(data)

    elif choice == 2:
        q.dequeue()

    elif choice == 3:
        q.peek()

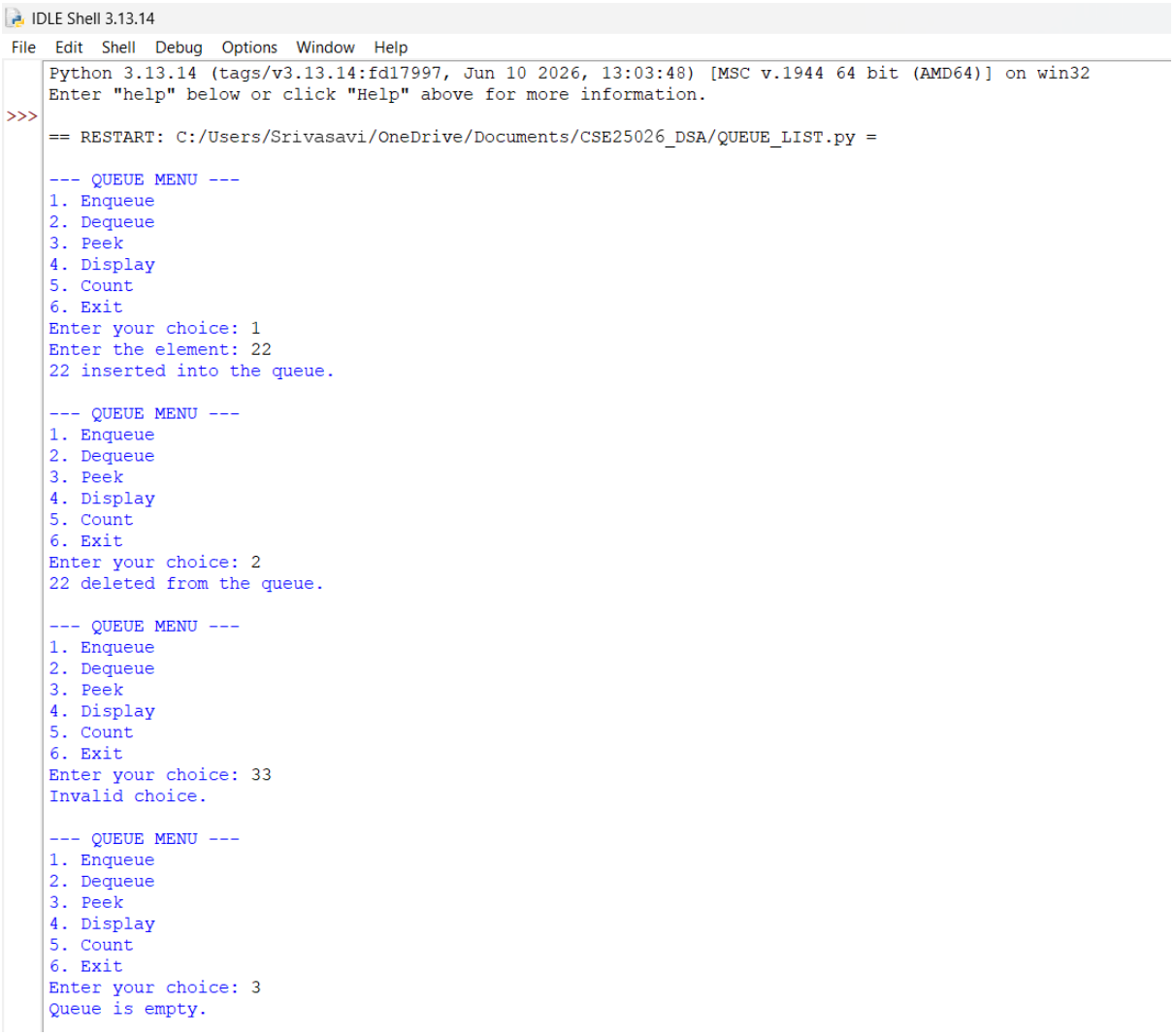
    elif choice == 4:
        q.display()

    elif choice == 5:
        q.count()

    elif choice == 6:
        print("Program ended.")
        break

    else:
        print("Invalid choice.")
```

Output:


IDLE Shell 3.13.14
File Edit Shell Debug Options Window Help
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
== RESTART: C:/Users/Srivasavi/OneDrive/Documents/CSE25026_DSA/QUEUE_LIST.py =
--- QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Count
6. Exit
Enter your choice: 1
Enter the element: 22
22 inserted into the queue.
--- QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Count
6. Exit
Enter your choice: 2
22 deleted from the queue.
--- QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Count
6. Exit
Enter your choice: 33
Invalid choice.
--- QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Count
6. Exit
Enter your choice: 3
Queue is empty.

```
--- QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Count
6. Exit
Enter your choice: 1
Enter the element: 222
222 inserted into the queue.

--- QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Count
6. Exit
Enter your choice: 1
Enter the element: 222
222 inserted into the queue.

--- QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Count
6. Exit
Enter your choice: 4
Queue elements are:
22
222
222

--- QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Count
6. Exit
Enter your choice: 6
Program ended.
```

Queue follows the **FIFO principle**:

- **FIFO = First In, First Out**
- The element inserted first is deleted first.

3. A queue can be implemented using an **array**.
4. Two variables are used to manage the queue:

- **Front** → points to the first element.
- **Rear** → points to the last element.

5. Initially, the queue is empty:
6. front = -1

rear = -1

7. **Enqueue** means inserting an element into the queue.
   - The element is inserted at the **rear**.
   - After insertion, rear is increased.
8. **Dequeue** means deleting an element from the queue.
   - The element is deleted from the **front**.
   - After deletion, front is increased.
9. **Peek** operation displays the element at the front without deleting it.
10. **Display** operation prints all the elements from front to rear.
11. **Count** operation gives the total number of elements in the queue.
12. **Queue Overflow** occurs when we try to insert an element when the array is full.
13. **Queue Underflow** occurs when we try to delete an element when the queue is empty.

6B)Aim: write a python program to implement the queue by using the linked list

Code:

QUEUE_LINKEDLIST.py - C:/Users/Srivasavi/OneDrive/Documents/CSE25026_DSA/WEEK-6/QUEUE_LINKEDLIST.py (3.13.14)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class Queue:
    def __init__(self):
        self.front = None
        self.rear = None

    # Enqueue operation
    def enqueue(self, data):
        new_node = Node(data)

        if self.rear is None:
            self.front = new_node
            self.rear = new_node
        else:
            self.rear.next = new_node
            self.rear = new_node

        print(data, "inserted into the queue.")

    # Dequeue operation
    def dequeue(self):
        if self.front is None:
            print("Queue Underflow. Queue is empty.")
        else:
            data = self.front.data
            self.front = self.front.next

            if self.front is None:
                self.rear = None

            print(data, "deleted from the queue.")

    # Peek operation
    def peek(self):
        if self.front is None:
            print("Queue is empty.")
        else:
            print("Front element is:", self.front.data)
```

```
    # Display all elements
    def display(self):
        if self.front is None:
            print("Queue is empty.")
        else:
            print("Queue elements are:")

            temp = self.front

            while temp is not None:
                print(temp.data)
                temp = temp.next

    # Count number of elements
    def count(self):
        count = 0
        temp = self.front

        while temp is not None:
            count += 1
            temp = temp.next

        print("Number of elements:", count)


# Main program
q = Queue()

while True:
    print("\n--- QUEUE MENU ---")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Count")
    print("6. Exit")

    choice = int(input("Enter your choice: "))
```

```
    if choice == 1:
        data = int(input("Enter the element: "))
        q.enqueue(data)

    elif choice == 2:
        q.dequeue()

    elif choice == 3:
        q.peek()

    elif choice == 4:
        q.display()

    elif choice == 5:
        q.count()

    elif choice == 6:
        print("Program ended.")
        break

    else:
        print("Invalid choice.")
```

```
# Main program
size = int(input("Enter the size of the circular queue: "))

q = CircularQueue(size)

while True:
    print("\n--- CIRCULAR QUEUE MENU ---")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Count")
    print("6. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        data = int(input("Enter the element: "))
        q.enqueue(data)

    elif choice == 2:
        q.dequeue()

    elif choice == 3:
        q.peek()

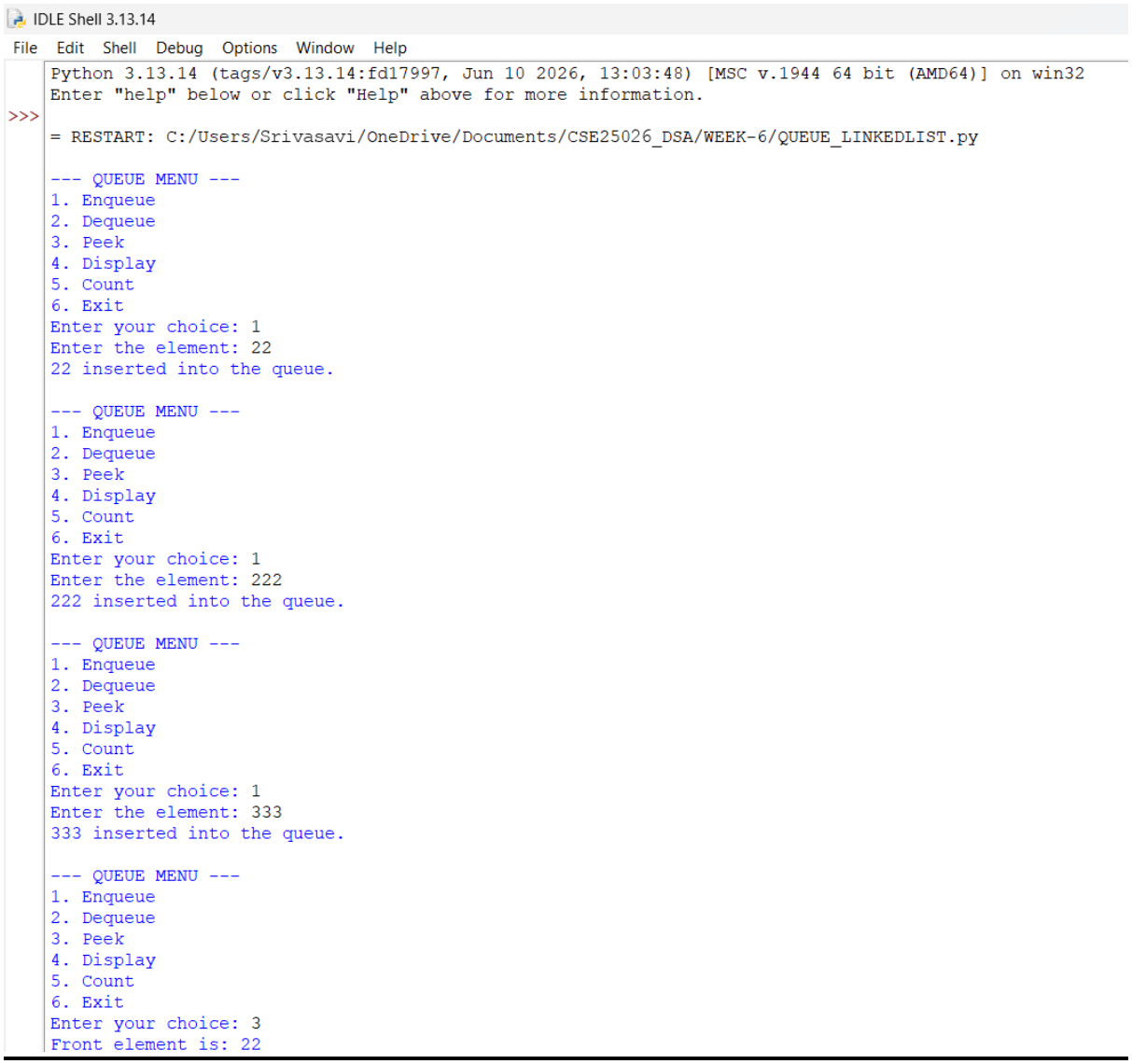
    elif choice == 4:
        q.display()

    elif choice == 5:
        q.count()

    elif choice == 6:
        print("Program ended.")
        break

    else:
        print("Invalid choice.")
```

Output:


IDLE Shell 3.13.14
File Edit Shell Debug Options Window Help
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/Srivasavi/OneDrive/Documents/CSE25026_DSA/WEEK-6/QUEUE_LINKEDLIST.py
--- QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Count
6. Exit
Enter your choice: 1
Enter the element: 22
22 inserted into the queue.
--- QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Count
6. Exit
Enter your choice: 1
Enter the element: 222
222 inserted into the queue.
--- QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Count
6. Exit
Enter your choice: 1
Enter the element: 333
333 inserted into the queue.
--- QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Count
6. Exit
Enter your choice: 3
Front element is: 22

```
--- QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Count
6. Exit
Enter your choice: 3
Front element is: 22

--- QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Count
6. Exit
Enter your choice: 4
Queue elements are:
22
222
333

--- QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Count
6. Exit
Enter your choice: 5
Number of elements: 3

--- QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Count
6. Exit
Enter your choice: 6
Program ended.
```

Explanation:

1. **Queue** is a linear data structure.
2. Queue follows the **FIFO principle**:
   - **FIFO = First In, First Out**
   - The element inserted first is deleted first.
3. A queue can be implemented using a **singly linked list**.
4. In a linked-list queue, each element is called a **node**.
5. Each node contains two parts:
   - **Data** → stores the element.

- **Next** → stores the address/reference of the next node.

6. Two pointers are used:

- **Front** → points to the first node.
- **Rear** → points to the last node.

7. Initially, the queue is empty:

8. front = None

rear = None

9. **Enqueue** means inserting an element into the queue.

- The new node is inserted at the **rear**.
- rear is updated to point to the new node.

10. **Dequeue** means deleting an element from the queue.

- The node is deleted from the **front**.
- front is moved to the next node.

11. If there is only **one node** in the queue and it is deleted:

- front = None
- rear = None

12. **Peek** operation displays the element at the front without deleting it.

13. **Display** operation traverses the linked list from front to None and displays all elements.

14. **Count** operation counts the number of nodes in the queue.

15. **Queue Underflow** occurs when we try to delete an element from an empty queue.

16. Unlike an array queue, a linked-list queue **does not have a fixed size**. It can grow as long as memory is available.

6c) Aim: write a python program to implement the circular queue by using the arrays

Code:

QUEUE_CIRCULAR.py - C:/Users/Srivasavi/OneDrive/Documents/CSE25026_DSA/WEEK-6/QUEUE_CIRCULAR.py (3.13.14)

File Edit Format Run Options Window Help

```
class CircularQueue:
    def __init__(self, size):
        self.size = size
        self.queue = [None] * size
        self.front = -1
        self.rear = -1

    # Enqueue operation
    def enqueue(self, data):
        # Check if queue is full
        if (self.rear + 1) % self.size == self.front:
            print("Queue Overflow. Queue is full.")
            return

        # First element
        if self.front == -1:
            self.front = 0
            self.rear = 0
        else:
            self.rear = (self.rear + 1) % self.size

        self.queue[self.rear] = data
        print(data, "inserted into the circular queue.")

    # Dequeue operation
    def dequeue(self):
        if self.front == -1:
            print("Queue Underflow. Queue is empty.")
            return

        data = self.queue[self.front]

        # Only one element is present
        if self.front == self.rear:
            self.front = -1
            self.rear = -1
        else:
            self.front = (self.front + 1) % self.size

        print(data, "deleted from the circular queue.")

    # Peek operation
    def peek(self):
        if self.front == -1:
            print("Queue is empty.")
        else:
            print("Front element is:", self.queue[self.front])
```

```
    # Peek operation
    def peek(self):
        if self.front == -1:
            print("Queue is empty.")
        else:
            print("Front element is:", self.queue[self.front])

    # Display operation
    def display(self):
        if self.front == -1:
            print("Queue is empty.")
        else:
            print("The elements of the queue are:")

            i = self.front

            while True:
                print(self.queue[i])

                if i == self.rear:
                    break

                i = (i + 1) % self.size

    # Count operation
    def count(self):
        if self.front == -1:
            print("Number of elements: 0")
        else:
            count = 0
            i = self.front

            while True:
                count += 1

                if i == self.rear:
                    break

                i = (i + 1) % self.size

            print("Number of elements:", count)
```

```
# Main program
size = int(input("Enter the size of the circular queue: "))

q = CircularQueue(size)

while True:
    print("\n--- CIRCULAR QUEUE MENU ---")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Count")
    print("6. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        data = int(input("Enter the element: "))
        q.enqueue(data)

    elif choice == 2:
        q.dequeue()

    elif choice == 3:
        q.peek()

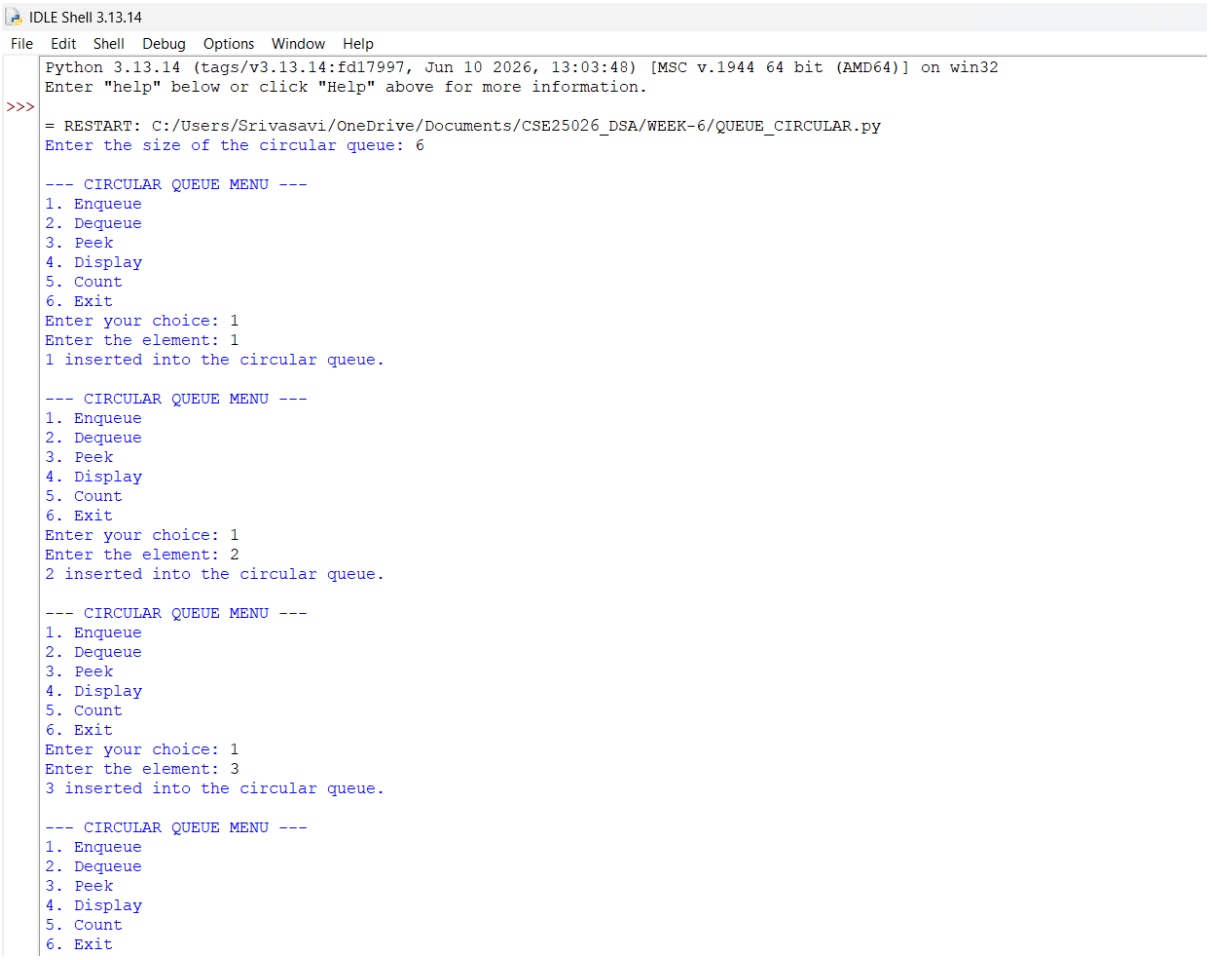
    elif choice == 4:
        q.display()

    elif choice == 5:
        q.count()

    elif choice == 6:
        print("Program ended.")
        break

    else:
        print("Invalid choice.")
```

Output:


IDLE Shell 3.13.14
File Edit Shell Debug Options Window Help
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/Srivasavi/OneDrive/Documents/CSE25026_DSA/WEEK-6/QUEUE_CIRCULAR.py
Enter the size of the circular queue: 6
--- CIRCULAR QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Count
6. Exit
Enter your choice: 1
Enter the element: 1
1 inserted into the circular queue.
--- CIRCULAR QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Count
6. Exit
Enter your choice: 1
Enter the element: 2
2 inserted into the circular queue.
--- CIRCULAR QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Count
6. Exit
Enter your choice: 1
Enter the element: 3
3 inserted into the circular queue.
--- CIRCULAR QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Count
6. Exit

```
--- CIRCULAR QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Count
6. Exit
Enter your choice: 1
Enter the element: 4
4 inserted into the circular queue.

--- CIRCULAR QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Count
6. Exit
Enter your choice: 4
The elements of the queue are:
1
2
3
4

--- CIRCULAR QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Count
6. Exit
Enter your choice: 5
Number of elements: 4

--- CIRCULAR QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Count
6. Exit
Enter your choice: 6
Program ended.
```

Explanation:

1. **Circular Queue** is a type of queue in which the last position of the array is connected back to the first position.
2. It follows the **FIFO principle**:
   - **FIFO = First In, First Out**
   - The element inserted first is deleted first.
3. A circular queue is implemented using an **array** of fixed size.

4. Two variables are used:
   - **Front** → points to the first element.
   - **Rear** → points to the last element.
5. Initially, the queue is empty:
6. front = -1

rear = -1

7. **Enqueue** means inserting an element into the queue.
   - The element is inserted at the **rear**.
   - rear is moved to the next position.
8. In a circular queue, when rear reaches the last index, it **wraps around to index 0**.
9. The circular movement is done using:

rear = (rear + 1) % size

10. **Dequeue** means deleting an element from the queue.
    - The element is deleted from the **front**.
    - front is moved to the next position.
11. The front also wraps around when it reaches the last index:

front = (front + 1) % size

11. **Peek** operation displays the front element without deleting it.
12. **Display** operation displays all elements starting from front and moving circularly until rear.
13. **Count** operation gives the number of elements currently present in the queue.
14. **Queue Overflow** occurs when the circular queue is full.
15. A common condition for a full circular queue is:

(rear + 1) % size == front

16. **Queue Underflow** occurs when the queue is empty.

WEEK-7

7a) **AIM:** Write a python program to implement the binary tree using arrays

Code:

7a.py - C:\Users\Srivasavi\OneDrive\Documents\CSE25026_DSA\WEEK-7\7a.py (3.13.14)

File Edit Format Run Options Window Help

```
class BinaryTree:
    def __init__(self, size=100):
        self.tree = [None] * size
        self.size = size

    def insert_root(self, value):
        if self.tree[0] is not None:
            print("Root already exists!")
        else:
            self.tree[0] = value
            print("Root inserted successfully.")

    def insert_left(self, parent, value):
        if parent not in self.tree:
            print("Parent not found!")
            return

        i = self.tree.index(parent)
        left = 2 * i + 1

        if left >= self.size:
            print("Array capacity exceeded!")
        elif self.tree[left] is not None:
            print("Left child already exists!")
        else:
            self.tree[left] = value
            print("Left child inserted successfully.")

    def insert_right(self, parent, value):
        if parent not in self.tree:
            print("Parent not found!")
            return

        i = self.tree.index(parent)
        right = 2 * i + 2

        if right >= self.size:
            print("Array capacity exceeded!")
        elif self.tree[right] is not None:
            print("Right child already exists!")
        else:
            self.tree[right] = value
            print("Right child inserted successfully.")
```

```
    def search(self, value):
        if value in self.tree:
            print("Element found.")
        else:
            print("Element not found.")

    def display(self):
        if all(x is None for x in self.tree):
            print("Tree is empty.")
            return

        print("Array representation:")
        for i, value in enumerate(self.tree):
            if value is not None:
                print(f"Index {i}: {value}")

    def preorder(self, i=0):
        if i >= self.size or self.tree[i] is None:
            return

        print(self.tree[i], end=" ")
        self.preorder(2 * i + 1)
        self.preorder(2 * i + 2)

    def inorder(self, i=0):
        if i >= self.size or self.tree[i] is None:
            return

        self.inorder(2 * i + 1)
        print(self.tree[i], end=" ")
        self.inorder(2 * i + 2)

    def postorder(self, i=0):
        if i >= self.size or self.tree[i] is None:
            return

        self.postorder(2 * i + 1)
        self.postorder(2 * i + 2)
        print(self.tree[i], end=" ")

    def delete(self, value):
        if value not in self.tree:
            print("Element not found!")
```

```
    def postorder(self, i=0):
        if i >= self.size or self.tree[i] is None:
            return

        self.postorder(2 * i + 1)
        self.postorder(2 * i + 2)
        print(self.tree[i], end=" ")

    def delete(self, value):
        if value not in self.tree:
            print("Element not found!")
            return

        i = self.tree.index(value)

        # A node can be deleted only if it has no children
        left = 2 * i + 1
        right = 2 * i + 2

        if ((left < self.size and self.tree[left] is not None)
                or (right < self.size and self.tree[right] is not None)):
            print("Delete its children first!")
        else:
            self.tree[i] = None
            print("Element deleted successfully.")


bt = BinaryTree()
```

```
while True:
    print("\n--- BINARY TREE MENU ---")
    print("1. Insert Root")
    print("2. Insert Left Child")
    print("3. Insert Right Child")
    print("4. Search Element")
    print("5. Display Tree")
    print("6. Preorder Traversal")
    print("7. Inorder Traversal")
    print("8. Postorder Traversal")
    print("9. Delete Element")
    print("10. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        value = int(input("Enter root value: "))
        bt.insert_root(value)

    elif choice == 2:
        parent = int(input("Enter parent value: "))
        value = int(input("Enter left child value: "))
        bt.insert_left(parent, value)

    elif choice == 3:
        parent = int(input("Enter parent value: "))
        value = int(input("Enter right child value: "))
        bt.insert_right(parent, value)

    elif choice == 4:
        value = int(input("Enter element to search: "))
        bt.search(value)

    elif choice == 5:
        bt.display()

    elif choice == 6:
        print("Preorder:", end=" ")
        bt.preorder()
        print()
```

```
    elif choice == 8:
        print("Postorder:", end=" ")
        bt.postorder()
        print()

    elif choice == 9:
        value = int(input("Enter element to delete: "))
        bt.delete(value)

    elif choice == 10:
        print("Exiting program.")
        break

    else:
        print("Invalid choice! Try again.")
```

Output:

```
>>>
== RESTART: C:\Users\Srivasavi\OneDrive\Documents\CSE25026_DSA\WEEK-7\7a.py ==

--- BINARY TREE MENU ---
1. Insert Root
2. Insert Left Child
3. Insert Right Child
4. Search Element
5. Display Tree
6. Preorder Traversal
7. Inorder Traversal
8. Postorder Traversal
9. Delete Element
10. Exit
Enter your choice: 1
Enter root value: 10
Root inserted successfully.
```

```
--- BINARY TREE MENU ---
1. Insert Root
2. Insert Left Child
3. Insert Right Child
4. Search Element
5. Display Tree
6. Preorder Traversal
7. Inorder Traversal
8. Postorder Traversal
9. Delete Element
10. Exit
Enter your choice: 2
Enter parent value: 20
Enter left child value: 15
Parent not found!
```

```
--- BINARY TREE MENU ---
1. Insert Root
2. Insert Left Child
3. Insert Right Child
4. Search Element
5. Display Tree
6. Preorder Traversal
7. Inorder Traversal
8. Postorder Traversal
9. Delete Element
10. Exit
Enter your choice: 3
Enter parent value: 10
Enter right child value: 15
Right child inserted successfully.
```

```
--- BINARY TREE MENU ---
1. Insert Root
2. Insert Left Child
3. Insert Right Child
4. Search Element
5. Display Tree
6. Preorder Traversal
7. Inorder Traversal
8. Postorder Traversal
9. Delete Element
10. Exit
Enter your choice: 4
Enter element to search: 13
Element not found.
```

```
--- BINARY TREE MENU ---
1. Insert Root
2. Insert Left Child
3. Insert Right Child
4. Search Element
5. Display Tree
6. Preorder Traversal
7. Inorder Traversal
8. Postorder Traversal
9. Delete Element
10. Exit
Enter your choice: 5
Array representation:
Index 0: 10
Index 2: 15
```

```
--- BINARY TREE MENU ---
1. Insert Root
2. Insert Left Child
3. Insert Right Child
4. Search Element
5. Display Tree
6. Preorder Traversal
7. Inorder Traversal
8. Postorder Traversal
9. Delete Element
10. Exit
Enter your choice: 6
Preorder: 10 15
```

```
--- BINARY TREE MENU ---
1. Insert Root
2. Insert Left Child
3. Insert Right Child
4. Search Element
5. Display Tree
6. Preorder Traversal
7. Inorder Traversal
8. Postorder Traversal
9. Delete Element
10. Exit
Enter your choice: 7
Inorder: 10 15
```

```
--- BINARY TREE MENU ---
1. Insert Root
2. Insert Left Child
3. Insert Right Child
4. Search Element
5. Display Tree
6. Preorder Traversal
7. Inorder Traversal
8. Postorder Traversal
9. Delete Element
10. Exit
Enter your choice: 8
Postorder: 15 10
```

```
--- BINARY TREE MENU ---
1. Insert Root
2. Insert Left Child
3. Insert Right Child
4. Search Element
5. Display Tree
6. Preorder Traversal
7. Inorder Traversal
8. Postorder Traversal
9. Delete Element
10. Exit
Enter your choice: 9
Enter element to delete: 15
Element deleted successfully.
```

```
--- BINARY TREE MENU ---
1. Insert Root
2. Insert Left Child
3. Insert Right Child
4. Search Element
5. Display Tree
6. Preorder Traversal
7. Inorder Traversal
8. Postorder Traversal
9. Delete Element
10. Exit
Enter your choice: 10
Exiting program.
```

Explanation:

1. A binary tree is a non-linear data structure in which each node can have at most two children: a left child and a right child.
2. In array representation, all nodes are stored in an array using index positions.
3. The root node is stored at index 0.
4. For a node at index i:
   - Left child: 2*i + 1
   - Right child: 2*i + 2
   - Parent: (i-1)//2, for i > 0.
5. Insertion: Inserts the root or a left/right child into the appropriate array position if it is empty.
6. Searching: Checks whether the given element exists in the array.
7. Display: Displays the tree elements along with their array indices.
8. Deletion: Deletes a leaf node by replacing its value with None.
9. Preorder traversal: Visits the root, left subtree, and right subtree.
10. Inorder traversal: Visits the left subtree, root, and right subtree.
11. Postorder traversal: Visits the left subtree, right subtree, and root.
12. Array representation provides direct index access but may waste memory for sparse trees.

2. Binary Tree Using a Linked List — Concept

1. A binary tree is a non-linear data structure in which each node has at most two children.
2. In linked-list representation, each node is created dynamically using a class.
3. Each node contains three components: data, left, and right.
4. The data field stores the value, while left and right store references to the child nodes.
5. The root variable points to the first node of the tree. If the tree is empty, the root is None.
6. Insertion: Creates a new node and connects it to the left or right of the specified parent node.
7. Searching: Uses a queue to search for the required element level by level.
8. Display: Displays all nodes in level-order traversal using a queue.
9. Deletion: Removes a leaf node by setting its parent's corresponding child reference to None.
10. Preorder traversal: Visits the root, left subtree, and right subtree.
11. Inorder traversal: Visits the left subtree, root, and right subtree.
12. Postorder traversal: Visits the left subtree, right subtree, and root.
13. Linked-list representation uses dynamic memory allocation and does not require a fixed array size.
14. It is suitable for irregularly shaped trees, but each node requires extra memory for references.

7b) **AIM:** write a python program to implement the binary tree using linked list

Code:

7b.py - C:\Users\Srivasavi\OneDrive\Documents\CSE25026_DSA\WEEK-7\7b.py (3.13.14)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None


class BinaryTree:
    def __init__(self):
        self.root = None

    def insert_root(self, value):
        if self.root is None:
            self.root = Node(value)
            print("Root inserted successfully.")
        else:
            print("Root already exists!")

    def find(self, value):
        if self.root is None:
            return None

        queue = [self.root]

        while queue:
            current = queue.pop(0)

            if current.data == value:
                return current

            if current.left:
                queue.append(current.left)

            if current.right:
                queue.append(current.right)

        return None

    def insert_left(self, parent, value):
        node = self.find(parent)

        if node is None:
            print("Parent not found!")
        elif node.left is not None:
            print("Left child already exists!")
        else:
            node.left = Node(value)
```

```
    def insert_right(self, parent, value):
        node = self.find(parent)

        if node is None:
            print("Parent not found!")
        elif node.right is not None:
            print("Right child already exists!")
        else:
            node.right = Node(value)
            print("Right child inserted successfully.")

    def search(self, value):
        if self.find(value):
            print("Element found.")
        else:
            print("Element not found.")

    def preorder(self, node):
        if node:
            print(node.data, end=" ")
            self.preorder(node.left)
            self.preorder(node.right)

    def inorder(self, node):
        if node:
            self.inorder(node.left)
            print(node.data, end=" ")
            self.inorder(node.right)

    def postorder(self, node):
        if node:
            self.postorder(node.left)
            self.postorder(node.right)
            print(node.data, end=" ")

    def display(self):
        if self.root is None:
            print("Tree is empty.")
            return

        print("Level-order display:")
        queue = [self.root]
```

```
        while queue:
            current = queue.pop(0)
            print(current.data, end=" ")

            if current.left:
                queue.append(current.left)

            if current.right:
                queue.append(current.right)

        print()

    def delete(self, value):
        if self.root is None:
            print("Tree is empty.")
            return

        if self.root.data == value:
            if self.root.left is None and self.root.right is None:
                self.root = None
                print("Element deleted successfully.")
            else:
                print("Delete its children first!")
            return

        queue = [self.root]

        while queue:
            current = queue.pop(0)

            if current.left:
                if current.left.data == value:
                    if (current.left.left is None and
                            current.left.right is None):
                        current.left = None
                        print("Element deleted successfully.")
                    else:
                        print("Delete its children first!")
                    return

                queue.append(current.left)

            if current.right:
                if current.right.data == value:
                    if (current.right.left is None and
```

```
            if current.right:
                if current.right.data == value:
                    if (current.right.left is None and
                            current.right.right is None):
                        current.right = None
                        print("Element deleted successfully.")
                    else:
                        print("Delete its children first!")
                    return

                queue.append(current.right)

        print("Element not found!")


bt = BinaryTree()

while True:
    print("\n--- BINARY TREE USING LINKED LIST ---")
    print("1. Insert Root")
    print("2. Insert Left Child")
    print("3. Insert Right Child")
    print("4. Search Element")
    print("5. Display Tree")
    print("6. Preorder Traversal")
    print("7. Inorder Traversal")
    print("8. Postorder Traversal")
    print("9. Delete Element")
    print("10. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        value = int(input("Enter root value: "))
        bt.insert_root(value)

    elif choice == 2:
        parent = int(input("Enter parent value: "))
        value = int(input("Enter left child value: "))
        bt.insert_left(parent, value)

    elif choice == 3:
        parent = int(input("Enter parent value: "))
        value = int(input("Enter right child value: "))
```

```
    elif choice == 4:
        value = int(input("Enter element to search: "))
        bt.search(value)

    elif choice == 5:
        bt.display()

    elif choice == 6:
        print("Preorder:", end=" ")
        bt.preorder(bt.root)
        print()

    elif choice == 7:
        print("Inorder:", end=" ")
        bt.inorder(bt.root)
        print()

    elif choice == 8:
        print("Postorder:", end=" ")
        bt.postorder(bt.root)
        print()

    elif choice == 9:
        value = int(input("Enter element to delete: "))
        bt.delete(value)

    elif choice == 10:
        print("Exiting program.")
        break

    else:
        print("Invalid choice! Try again.")
```

Output:

```
>>>
== RESTART: C:/Users/Srivasavi/OneDrive/Documents/CSE25026_DSA/WEEK-7/7b.py ==

--- BINARY TREE USING LINKED LIST ---
1. Insert Root
2. Insert Left Child
3. Insert Right Child
4. Search Element
5. Display Tree
6. Preorder Traversal
7. Inorder Traversal
8. Postorder Traversal
9. Delete Element
10. Exit
Enter your choice: 1
Enter root value: 10
Root inserted successfully.
```

```
--- BINARY TREE USING LINKED LIST ---
1. Insert Root
2. Insert Left Child
3. Insert Right Child
4. Search Element
5. Display Tree
6. Preorder Traversal
7. Inorder Traversal
8. Postorder Traversal
9. Delete Element
10. Exit
Enter your choice: 2
Enter parent value: 12
Enter left child value: 12
Parent not found!
```

```
--- BINARY TREE USING LINKED LIST ---
1. Insert Root
2. Insert Left Child
3. Insert Right Child
4. Search Element
5. Display Tree
6. Preorder Traversal
7. Inorder Traversal
8. Postorder Traversal
9. Delete Element
10. Exit
Enter your choice: 3
Enter parent value: 10
Enter right child value: 3
Right child inserted successfully.
```

```
--- BINARY TREE USING LINKED LIST ---
1. Insert Root
2. Insert Left Child
3. Insert Right Child
4. Search Element
5. Display Tree
6. Preorder Traversal
7. Inorder Traversal
8. Postorder Traversal
9. Delete Element
10. Exit
Enter your choice: 4
Enter element to search: 13
Element not found.
```

```
--- BINARY TREE USING LINKED LIST ---
1. Insert Root
2. Insert Left Child
3. Insert Right Child
4. Search Element
5. Display Tree
6. Preorder Traversal
7. Inorder Traversal
8. Postorder Traversal
9. Delete Element
10. Exit
Enter your choice: 5
Level-order display:
10 3
```

```
--- BINARY TREE USING LINKED LIST ---
1. Insert Root
2. Insert Left Child
3. Insert Right Child
4. Search Element
5. Display Tree
6. Preorder Traversal
7. Inorder Traversal
8. Postorder Traversal
9. Delete Element
10. Exit
Enter your choice: 6
Preorder: 10 3
```

```
--- BINARY TREE USING LINKED LIST ---
1. Insert Root
2. Insert Left Child
3. Insert Right Child
4. Search Element
5. Display Tree
6. Preorder Traversal
7. Inorder Traversal
8. Postorder Traversal
9. Delete Element
10. Exit
Enter your choice: 7
Inorder: 10 3
```

```
--- BINARY TREE USING LINKED LIST ---
1. Insert Root
2. Insert Left Child
3. Insert Right Child
4. Search Element
5. Display Tree
6. Preorder Traversal
7. Inorder Traversal
8. Postorder Traversal
9. Delete Element
10. Exit
Enter your choice: 8
Postorder: 3 10
```

```
--- BINARY TREE USING LINKED LIST ---
1. Insert Root
2. Insert Left Child
3. Insert Right Child
4. Search Element
5. Display Tree
6. Preorder Traversal
7. Inorder Traversal
8. Postorder Traversal
9. Delete Element
10. Exit
Enter your choice: 9
Enter element to delete: 2
Element not found!
```

```
--- BINARY TREE USING LINKED LIST ---
1. Insert Root
2. Insert Left Child
3. Insert Right Child
4. Search Element
5. Display Tree
6. Preorder Traversal
7. Inorder Traversal
8. Postorder Traversal
9. Delete Element
10. Exit
Enter your choice: 10
Exiting program.
```

Explanation:

1. A binary tree is a non-linear data structure in which each node can have at most two children: a left child and a right child.

2. In linked-list representation, each node is created dynamically using a class.

3. Each node consists of three components: data, left, and right.

4. The data field stores the value of the node, while left and right store references to its child nodes.

5. The root variable points to the first node of the binary tree. If the tree is empty, the root is set to None.

6. Insertion: A new node is created and connected as the left or right child of the specified parent node.

7. Searching: The program searches for a given element using level-order traversal with a queue.

8. Display: The elements of the binary tree are displayed in level order, starting from the root node.

9. Deletion: A leaf node is deleted by setting the corresponding reference of its parent to None.

10. Preorder traversal: Visits the root node, left subtree, and right subtree.

11. Inorder traversal: Visits the left subtree, root node, and right subtree.

12. Postorder traversal: Visits the left subtree, right subtree, and root node.

13. Linked-list representation allocates memory dynamically and does not require a fixed-size array.

14. It is suitable for trees of different shapes, but each node requires additional memory to store references.

Error table:

sno	Error obtained	Error rectification
1	Searching for an element that does not exist in the tree.	Added conditions to display an appropriate message when the element is not found
2	Incorrect traversal of tree elements.	Implemented inorder, preorder, and postorder

		traversal functions using recursion.

Week-8

Aim: write a python program to implement the binary search tree using the arrays

Code:

8a.py - C:\Users\Srivasavi\OneDrive\Documents\CSE25026_DSA\WEEk-8\8a.py (3.13.14)

File Edit Format Run Options Window Help

```
tree = []


def insert(value):
    global tree

    if len(tree) == 0:
        tree.append(value)
        print("Element inserted successfully")
        return

    i = 0

    while i < len(tree):
        if tree[i] is None:
            tree[i] = value
            print("Element inserted successfully")
            return

        elif value < tree[i]:
            i = 2 * i + 1

        elif value > tree[i]:
            i = 2 * i + 2

        else:
            print("Duplicate elements are not allowed")
            return

    while i >= len(tree):
        tree.append(None)

    tree[i] = value
    print("Element inserted successfully")


def search(value):
    i = 0

    while i < len(tree):
        if tree[i] is None:
            print("Element not found")
            return
```

```
def search(value):
    i = 0

    while i < len(tree):
        if tree[i] is None:
            print("Element not found")
            return

        elif tree[i] == value:
            print("Element found")
            return

        elif value < tree[i]:
            i = 2 * i + 1

        else:
            i = 2 * i + 2

    print("Element not found")


def inorder(i=0):
    if i < len(tree) and tree[i] is not None:
        inorder(2 * i + 1)
        print(tree[i], end=" ")
        inorder(2 * i + 2)


def preorder(i=0):
    if i < len(tree) and tree[i] is not None:
        print(tree[i], end=" ")
        preorder(2 * i + 1)
        preorder(2 * i + 2)


def postorder(i=0):
    if i < len(tree) and tree[i] is not None:
        postorder(2 * i + 1)
        postorder(2 * i + 2)
        print(tree[i], end=" ")
```

```
def display():
    if len(tree) == 0 or tree[0] is None:
        print("Tree is empty")
    else:
        print("Array representation:", tree)


def delete(value):
    global tree

    def find_index(value):
        i = 0

        while i < len(tree):
            if tree[i] is None:
                return -1

            elif tree[i] == value:
                return i

            elif value < tree[i]:
                i = 2 * i + 1

            else:
                i = 2 * i + 2

        return -1

    index = find_index(value)

    if index == -1:
        print("Element not found")
        return

    elements = [
        tree[i]
        for i in range(len(tree))
        if tree[i] is not None and i != index
    ]

    tree = []

    for element in elements:
        insert(element)
```

```
while True:
    print("\n--- BINARY SEARCH TREE MENU ---")
    print("1. Insert")
    print("2. Search")
    print("3. Delete")
    print("4. Inorder Traversal")
    print("5. Preorder Traversal")
    print("6. Postorder Traversal")
    print("7. Display")
    print("8. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        value = int(input("Enter element to insert: "))
        insert(value)

    elif choice == 2:
        value = int(input("Enter element to search: "))
        search(value)

    elif choice == 3:
        value = int(input("Enter element to delete: "))
        delete(value)

    elif choice == 4:
        print("Inorder Traversal:", end=" ")
        inorder()
        print()

    elif choice == 5:
        print("Preorder Traversal:", end=" ")
        preorder()
        print()

    elif choice == 6:
        print("Postorder Traversal:", end=" ")
        postorder()
        print()

    elif choice == 7:
        display()

    elif choice == 8:
        print("Program terminated")
        break
```

Output:

```
>>>
== RESTART: C:/Users/Srivasavi/OneDrive/Documents/CSE25026_DSA/WEEk-8/8a.py ==

--- BINARY SEARCH TREE MENU ---
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display
8. Exit
Enter your choice: 1
Enter element to insert: 23
Element inserted successfully
```

```
--- BINARY SEARCH TREE MENU ---
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display
8. Exit
Enter your choice: 1
Enter element to insert: 24
Element inserted successfully
```

```
--- BINARY SEARCH TREE MENU ---
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display
8. Exit
Enter your choice: 1
Enter element to insert: 56
Element inserted successfully
```

```
--- BINARY SEARCH TREE MENU ---
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display
8. Exit
Enter your choice: 1
Enter element to insert: 75
Element inserted successfully
```

```
--- BINARY SEARCH TREE MENU ---
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display
8. Exit
Enter your choice: 1
Enter element to insert: 65
Element inserted successfully
```

```
--- BINARY SEARCH TREE MENU ---
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display
8. Exit
Enter your choice: 2
Enter element to search: 45
Element not found
```

```
--- BINARY SEARCH TREE MENU ---
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display
8. Exit
Enter your choice: 3
Enter element to delete: 56
Element inserted successfully
Element inserted successfully
Element inserted successfully
Element inserted successfully
Element deleted successfully
```

```
--- BINARY SEARCH TREE MENU ---
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display
8. Exit
Enter your choice: 4
Inorder Traversal: 23 24 65 75
```

```
--- BINARY SEARCH TREE MENU ---
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display
8. Exit
Enter your choice: 6
Postorder Traversal: 65 75 24 23
```

```
--- BINARY SEARCH TREE MENU ---
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display
8. Exit
Enter your choice: 7
Array representation: [23, None, 24, None, None, None, 75, None, None, None, None, None, None, 65]
```

```
--- BINARY SEARCH TREE MENU ---
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display
8. Exit
Enter your choice: 8
Program terminated
```

Explanation:

1. **Initialization:** An empty list named tree is created to store the elements of the Binary Search Tree (BST).
2. **Insertion Operation:** The insert() function inserts a new element into the tree. If the value is smaller than the current node, it moves to the left child. If the value is greater, it moves to the right child. Duplicate elements are not allowed.
3. **Array Representation:** The tree is represented using an array. For a node at index i, the left child is located at 2*i + 1, and the right child is located at 2*i + 2.
4. **Search Operation:** The search() function searches for a given element. It compares the value with the current node and moves to the left or right child until the element is found or the search fails.
5. **Deletion Operation:** The delete() function removes the specified element from the tree. It collects the remaining elements and rebuilds the BST without the deleted element.
6. **Inorder Traversal:** The inorder() function visits the left subtree, root node, and right subtree. It displays the elements in ascending order.
7. **Preorder Traversal:** The preorder() function visits the root node first, followed by the left subtree and the right subtree.
8. **Postorder Traversal:** The postorder() function visits the left subtree, the right subtree, and finally the root node.

9. **Display Operation:** The display() function displays the array representation of the BST. If the tree is empty, it displays an appropriate message.
10. **Menu-Driven Interface:** The program displays a menu containing all the available operations. The user selects an operation by entering the corresponding choice.
11. **Exit Operation:** When the user selects option 8, the program displays a termination message and stops executing.
12. **Invalid Choice Handling:** If the user enters a choice that is not available in the menu, the program displays an invalid-choice message and allows the user to try again.
13. **Program Termination:** The while True loop repeatedly displays the menu and performs the selected operation until the user chooses to exit.

Top of Form

Bottom of Form

error table:

Sno	Error obtained	Error rectification
1	Incorrect array indexing while inserting elements.	Used the formulas 2*i + 1 for the left child and 2*i + 2 for the right child.
2	Duplicate elements may be inserted into the tree.	Added a condition to check for duplicate elements before insertion.

Week-9

Aim: write a python program to implement the AVL tree using arrays

Code:

AVL Tree.py - C:/Users/Srivasavi/OneDrive/Documents/CSE25026_DSA/AVL Tree.py (3.13.14)

File Edit Format Run Options Window Help

```
class AVLTree:
    def __init__(self):
        self.keys = []
        self.left = []
        self.right = []
        self.height = []
        self.root = -1

    # Create a new node
    def create_node(self, key):
        i = len(self.keys)
        self.keys.append(key)
        self.left.append(-1)
        self.right.append(-1)
        self.height.append(1)
        return i

    # Get height and balance factor
    def get_height(self, i):
        return 0 if i == -1 else self.height[i]

    def balance(self, i):
        if i == -1:
            return 0
        return self.get_height(self.left[i]) - self.get_height(self.right[i])

    # Update height
    def update_height(self, i):
        self.height[i] = 1 + max(
            self.get_height(self.left[i]),
            self.get_height(self.right[i])
        )

    # Right rotation
    def right_rotate(self, y):
        x = self.left[y]
        t2 = self.right[x]

        self.right[x] = y
        self.left[y] = t2

        self.update_height(y)
        self.update_height(x)
        return x
```

```
    # Left rotation
    def left_rotate(self, x):
        y = self.right[x]
        t2 = self.left[y]

        self.left[y] = x
        self.right[x] = t2

        self.update_height(x)
        self.update_height(y)
        return y

    # Balance the tree
    def rebalance(self, i, key=None):
        self.update_height(i)
        bf = self.balance(i)

        # LL case
        if bf > 1 and self.balance(self.left[i]) >= 0:
            return self.right_rotate(i)

        # RR case
        if bf < -1 and self.balance(self.right[i]) <= 0:
            return self.left_rotate(i)

        # LR case
        if bf > 1 and self.balance(self.left[i]) < 0:
            self.left[i] = self.left_rotate(self.left[i])
            return self.right_rotate(i)

        # RL case
        if bf < -1 and self.balance(self.right[i]) > 0:
            self.right[i] = self.right_rotate(self.right[i])
            return self.left_rotate(i)

        return i
```

```
    # Insert
    def insert_node(self, i, key):
        if i == -1:
            return self.create_node(key)

        if key < self.keys[i]:
            self.left[i] = self.insert_node(self.left[i], key)
        elif key > self.keys[i]:
            self.right[i] = self.insert_node(self.right[i], key)
        else:
            print("Duplicate values are not allowed")
            return i

        return self.rebalance(i)

    # Search
    def search(self, i, key):
        if i == -1:
            return False

        if self.keys[i] == key:
            return True

        if key < self.keys[i]:
            return self.search(self.left[i], key)

        return self.search(self.right[i], key)

    # Find minimum node
    def min_node(self, i):
        while self.left[i] != -1:
            i = self.left[i]
        return i

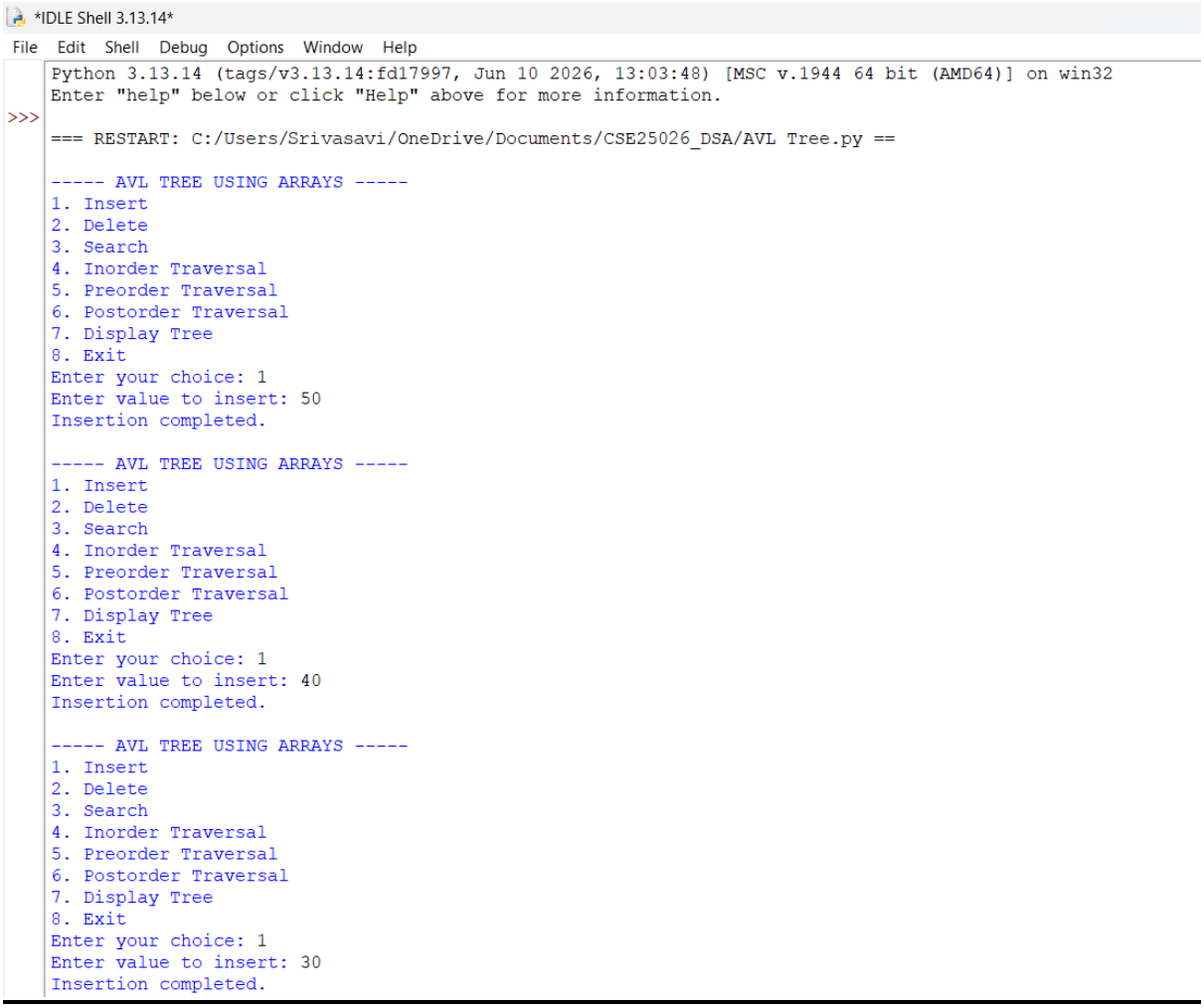
    # Delete
    def delete_node(self, i, key):
        if i == -1:
            return -1

        if key < self.keys[i]:
            self.left[i] = self.delete_node(self.left[i], key)

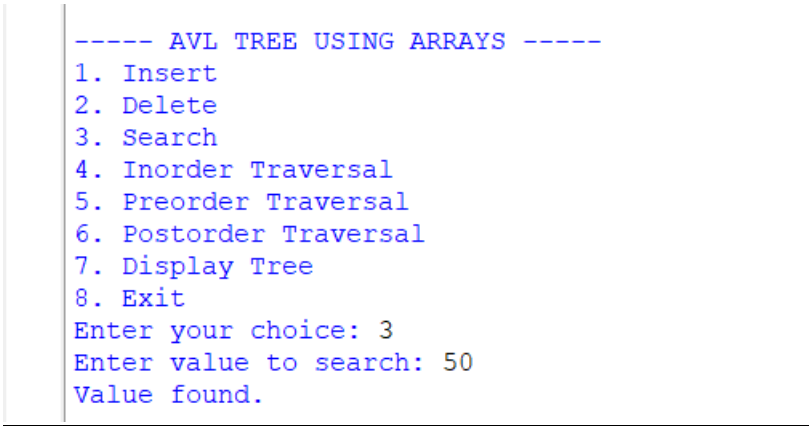
        elif key > self.keys[i]:
            self.right[i] = self.delete_node(self.right[i], key)
```

Output:

Insert element



Search element



Delete element

```
----- AVL TREE USING ARRAYS -----
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display Tree
8. Exit
Enter your choice: 2
Enter value to delete: 40
Node deleted successfully.

----- AVL TREE USING ARRAYS -----
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display Tree
8. Exit
Enter your choice: 2
Enter value to delete: 40
Value not found.
```

Display tree

```
----- AVL TREE USING ARRAYS -----
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display Tree
8. Exit
Enter your choice: 7
      6543
   2345
50
   30
```

Inorder traversal \ Preorder traversal \ Postorder traversal

```
----- AVL TREE USING ARRAYS -----
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display Tree
8. Exit
Enter your choice: 4
Inorder: 30 50 2345 6543

----- AVL TREE USING ARRAYS -----
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display Tree
8. Exit
Enter your choice: 5
Preorder: 50 30 2345 6543

----- AVL TREE USING ARRAYS -----
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display Tree
8. Exit
Enter your choice: 6
Postorder: 30 6543 2345 50
```

exit

```
----- AVL TREE USING ARRAYS -----
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display Tree
8. Exit
Enter your choice: 8
Exiting program.
>>>
```

Explanation:

- The program implements an AVL tree using arrays in Python.
- The keys, left, right, and height arrays store node values, child indices, and heights.
- The create_node() function creates a new node and initializes its values.
- The get_height() function returns the height of a node.
- The update_height() function updates the height of a node.
- The balance() function calculates the balance factor of a node.
- The right_rotate() function performs a right rotation to balance the tree.
- The left_rotate() function performs a left rotation to balance the tree.
- The insert_node() function inserts new values and balances the tree when necessary.
- The program handles four imbalance cases: LL, RR, LR, and RL.
- The search() function checks whether a value exists in the tree.
- The min_node() function finds the minimum value in a subtree.
- The delete_node() function deletes a node and restores the tree's balance.
- The inorder() function displays values in ascending order.
- The preorder() function visits the root before its children.
- The postorder() function visits the root after its children.
- The display() function displays the tree structure.
- The main program provides a menu for insertion, deletion, searching, traversals, and displaying the tree.
- The try-except block handles invalid integer inputs.
- The program maintains the AVL tree's balance to make searching, insertion, and deletion efficient.

Error table:

sno	Error obtained	Error rectification
1	Incorrect balance factor	Calculated the balance factor as left subtree height minus right subtree height.
2	Root index not updated after rotation	Stored the returned root index in avl.root.

Week-10

Aim: write a python program to hashing

Code:

WEEK-10.py - C:/Users/Srivasavi/OneDrive/Documents/CSE25026_DSA/WEEK-10.py (3.13.14)

File Edit Format Run Options Window Help

```
SIZE = 10

# Separate Chaining
chain = [[] for _ in range(SIZE)]

# Linear Probing
linear = [None] * SIZE

# Quadratic Probing
quadratic = [None] * SIZE


# Hash function
def hash_func(key):
    return key % SIZE


# Separate Chaining
def chaining(key):
    index = hash_func(key)
    if key not in chain[index]:
        chain[index].append(key)
        print("Inserted successfully.")
    else:
        print("Duplicate key!")


# Linear Probing
def linear_probing(key):
    index = hash_func(key)

    for i in range(SIZE):
        pos = (index + i) % SIZE

        if linear[pos] == key:
            print("Duplicate key!")
            return

        if linear[pos] is None:
            linear[pos] = key
            print("Inserted successfully.")
            return

    print("Hash table is full.")
```

```
# Quadratic Probing
def quadratic_probing(key):
    index = hash_func(key)

    for i in range(SIZE):
        pos = (index + i * i) % SIZE

        if quadratic[pos] == key:
            print("Duplicate key!")
            return

        if quadratic[pos] is None:
            quadratic[pos] = key
            print("Inserted successfully.")
            return

    print("No empty position found.")


# Search
def search(table, key, method):
    index = hash_func(key)

    if method == 1:  # Chaining
        return key in table[index]

    for i in range(SIZE):
        if method == 2:
            pos = (index + i) % SIZE
        else:
            pos = (index + i * i) % SIZE

        if table[pos] is None:
            return False

        if table[pos] == key:
            return True

    return False
```

```
# Display
def display(table, method):
    for i in range(SIZE):
        print(i, ":", table[i])


# Main program
while True:
    print("\n----- HASHING -----")
    print("1. Separate Chaining")
    print("2. Linear Probing")
    print("3. Quadratic Probing")
    print("4. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 4:
        print("Exiting program.")
        break

    if choice not in [1, 2, 3]:
        print("Invalid choice!")
        continue

    table = [chain, linear, quadratic][choice - 1]

    while True:
        print("\n1. Insert")
        print("2. Search")
        print("3. Display")
        print("4. Back to Main Menu")

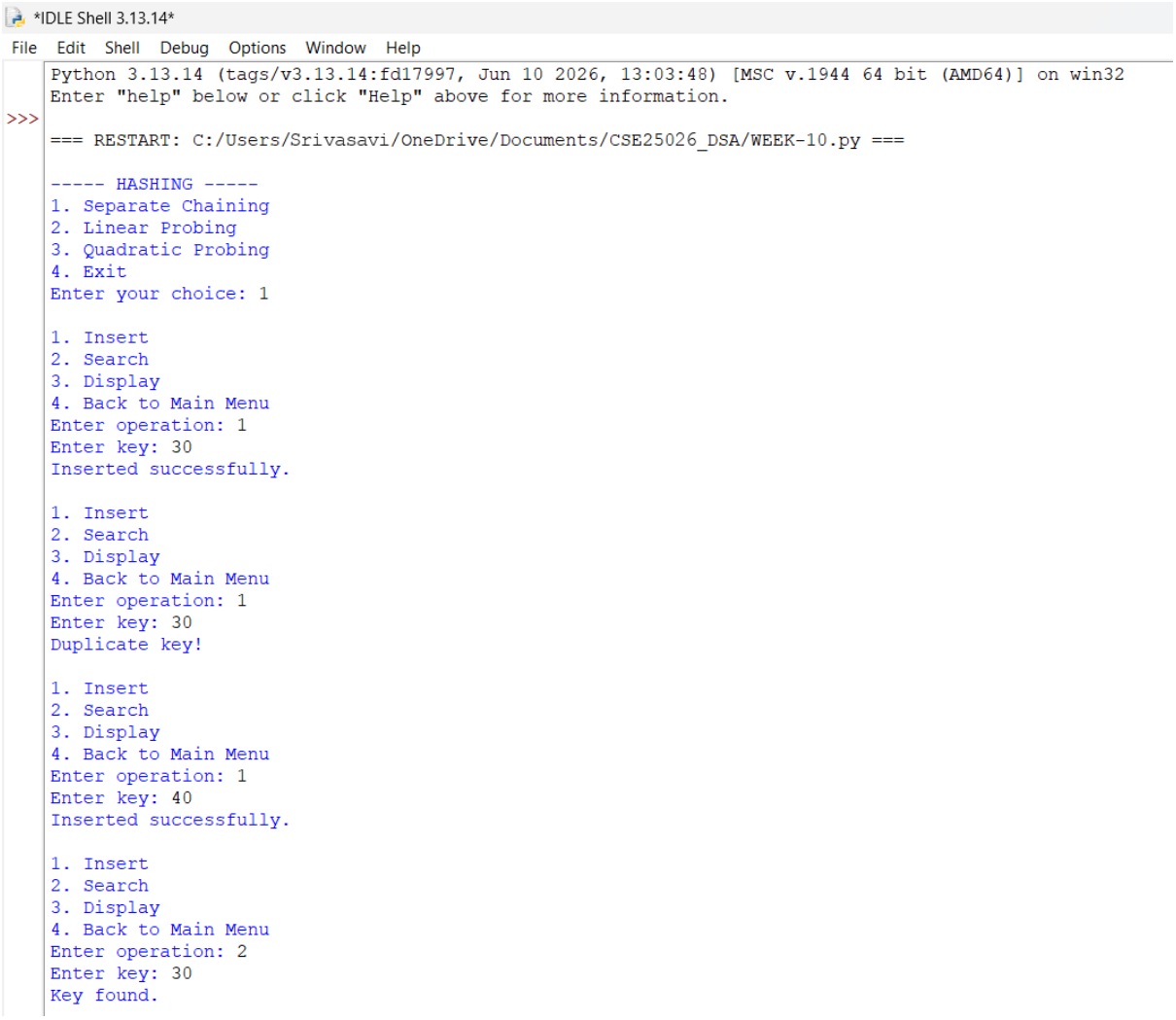
        op = int(input("Enter operation: "))

        if op == 1:
            key = int(input("Enter key: "))

            if choice == 1:
                chaining(key)
            elif choice == 2:
                linear_probing(key)
            else:
                quadratic_probing(key)
```

Output:

Separate Chaining:



```
1. Insert
2. Search
3. Display
4. Back to Main Menu
Enter operation: 1
Enter key: 40
Inserted successfully.

1. Insert
2. Search
3. Display
4. Back to Main Menu
Enter operation: 2
Enter key: 30
Key found.

1. Insert
2. Search
3. Display
4. Back to Main Menu
Enter operation: 3
0 : [30, 40]
1 : []
2 : []
3 : []
4 : []
5 : []
6 : []
7 : []
8 : []
9 : []

1. Insert
2. Search
3. Display
4. Back to Main Menu
Enter operation: 4

----- HASHING -----
1. Separate Chaining
2. Linear Probing
3. Quadratic Probing
4. Exit
Enter your choice:
```

Quadratic Probing:

```
----- HASHING -----
1. Separate Chaining
2. Linear Probing
3. Quadratic Probing
4. Exit
Enter your choice: 3

1. Insert
2. Search
3. Display
4. Back to Main Menu
Enter operation: 1
Enter key: 10
Inserted successfully.

1. Insert
2. Search
3. Display
4. Back to Main Menu
Enter operation: 10
Invalid operation!

1. Insert
2. Search
3. Display
4. Back to Main Menu
Enter operation: 3
0 : 10
1 : None
2 : None
3 : None
4 : None
5 : None
6 : None
7 : None
8 : None
9 : None

1. Insert
2. Search
3. Display
4. Back to Main Menu
Enter operation:
```

Linear Probing:

```
1. Insert
2. Search
3. Display
4. Back to Main Menu
Enter operation: 1
Enter key: 1
Inserted successfully.

1. Insert
2. Search
3. Display
4. Back to Main Menu
Enter operation: 1
Enter key: 2
Inserted successfully.

1. Insert
2. Search
3. Display
4. Back to Main Menu
Enter operation: 1
Enter key: 2
Duplicate key!

1. Insert
2. Search
3. Display
4. Back to Main Menu
Enter operation: 3
0 : None
1 : 1
2 : 2
3 : None
4 : None
5 : None
6 : None
7 : None
8 : None
9 : None

1. Insert
2. Search
3. Display
4. Back to Main Menu
Enter operation:
```

Explanation:

- The program implements three hashing techniques: Separate Chaining, Linear Probing, and Quadratic Probing.
- The SIZE = 11 statement defines the size of the hash table as 11.
- The chaining list stores elements using separate chaining.
- The linear list stores elements using linear probing.
- The quadratic list stores elements using quadratic probing.

- The insert_chaining() function inserts an element into the corresponding index using the modulo operation.
- The search_chaining() function searches for an element in the corresponding chain.
- The display_chaining() function displays all the elements stored in the separate chaining hash table.
- The insert_linear() function inserts an element by checking consecutive positions when a collision occurs.
- The search_linear() function searches for an element using linear probing.
- The display_linear() function displays the elements in the linear probing hash table.
- The insert_quadratic() function resolves collisions by checking positions using the quadratic formula.
- The search_quadratic() function searches for an element using quadratic probing.
- The display_quadratic() function displays the elements in the quadratic probing hash table.
- The modulo operation key % SIZE calculates the initial hash index.
- The None value indicates an empty position in the linear and quadratic probing tables.
- The main menu allows the user to select a hashing technique or exit the program.
- The submenu allows the user to insert, search, display elements, or return to the main menu.
- The if-elif-else statements execute the selected operation.
- The program demonstrates how collisions are handled using three different hashing techniques.

Error table:

Sno	Error obtained	Error rectification
1	Hash table becomes full during insertion.	Used separate chaining, linear probing and quadratic probing to resolve collisions.
2	Duplicate elements are inserted into the hash table.	Added a condition to check whether the key already exists..
3.	Search operation gives an incorrect result.	Corrected the probing sequence during searching.